

Supplementary information: Collective cooperation without individual fidelity in LLM agents

Henrique Ferraz de Arruda^{1,2}, Carlos Gracia Lázaro³, Alberto Aleta^{2,4}, and Yamir Moreno^{2,4}

¹ARAID Foundation, Zaragoza, Spain

²Institute for Biocomputation and Physics of Complex Systems, University of Zaragoza, Zaragoza, Spain

³Universidad San Jorge (USJ), Zaragoza, Spain

⁴Department of Theoretical Physics, University of Zaragoza, Zaragoza, Spain

June 4, 2026

S1 Technical details

S1.1 Overview

We simulate a repeated multi-agent repeated Prisoner’s Dilemma on an input network. Each node is controlled either by an LLM-based agent or by a randomized policy. The CrewAI orchestration layer is used to orchestrate LLM agents, tasks, and responses, as well as an LLM served via an Ollama-compatible endpoint.

In summary, in each round of the dynamics:

1. Each agent receives a brief description of the previous round, including neighbors’ choices and normalized payoffs.
2. Each agent replies with a JSON object containing a **choice** (“GREEN” or “BROWN”) and a free-text *reasoning*.
3. The simulation computes payoffs for every node based on the payoff matrix, logs the answers, and proceeds to the next round.

As for the graph input and experimental setup, we used the same graphs used in the original paper (i.e., [2]). This includes the network structures and the temporal changes of edges for the control cases, as well as numbers of iterations.

S1.2 Agent creation and random agents

For each node, we create either:

- an LLM-driven **Agent** (CrewAI), or
- a random agent (chooses uniformly between GREEN and BROWN).

A fraction ρ of nodes are forced to be random. These nodes remain random throughout the dynamics’ execution.

S1.3 Payoff computation

Payoffs are pairwise and defined by the following matrix (per neighbor):

You / Neighbor	GREEN	BROWN
GREEN	7	0
BROWN	10	0

The node's payoff in a round equals the sum over its neighbors of the payoff according to each neighbor's choice.

Implementation details:

- It iterates over neighbors of node i and sums the correct payoff depending on the node's choice and the neighbor's choice.
- For comparison purposes, a *normalized* neighbor payoff is computed when exposing neighbor payoffs to agents. The normalization rescales a neighbor's payoff by the ratio of the focal node's degree to the neighbor's degree.

S1.4 LLM orchestration and prompts

Important fields passed to each **Agent** instance include:

- **role**: short role description shown to the model.
- **goal**: short task: "maximize your own ECUs." ECU is a monetary unit.
- **backstory**: personalized per agent (degree and optional backstory fields).
- **system template**: a brief system message explaining the experiment.
- **prompt template**: in the implementation set to:

Answer the task below in valid JSON format: {input}

The placeholder {input} is a dynamic variable that is automatically replaced at runtime with the task-specific description passed to the **Task** object.

- The CrewAI **Task** is created with expected output describing the required JSON format.

The main LLM configuration values used by the script are:

- `llm model = 'ollama/llama4:16x17b'` (default, but we also tested several LLMs).

S1.4.1 Prompts and textual templates

The textual templates used are reproduced below. These are sent to the agent together with the task description.

Role and goal

```
role = "You are participating in a social experiment as a volunteer. Your objective is to maximize your total ECUs, the monetary unit. An ECU is a monetary unit"
```

```
goal = "Objective: **Your goal is to maximize your own ECUs (ECU is a monetary unit).**\n"
```

System prompt

```
system_template = "You are not expected to behave in any particular way. Whatever you do will determine the amount of money you can earn. You have a written version of this direction, which you can check at any stage of the experiment."
```

Prompt template

```
"Answer the task below in valid JSON format: {input}"
```

Expected response format

All agents must reply with a JSON object in the following format (this is enforced using the `expected_output` string in the code):

```
String in a Json format ONLY with choice (GREEN or BROWN) and reasoning (plain text):
{
  "choice": ...,
  "reasoning": "..."
}
```

First-iteration description (displayed to the agent at round 0)

A human-readable explanation of payoffs and examples is provided.

Possible payoffs per neighbor.

In the following table each row corresponds to the decision you can make and each column correspond to one of your neighbors' decision.

Your choice / Neighbor choice \t GREEN \t BROWN

GREEN \t 7 \t 0

BROWN \t 10 \t 0

Consider that: you and each of your neighbors (which are humans) will globally earn more if you both choose GREEN (7 ECUs you/7 ECUs your neighbor); you will earn more if you choose BROWN and your neighbor chooses GREEN (10 ECUs you/0 ECUs your neighbor); but if both you and your neighbor choose BROWN you both will earn less (0 ECUs you/0 ECUs your neighbor) than if you both chose GREEN.

This is the screen you will be seeing during the experiment (note that each participant actually sees the graph corresponding to his/her connectivity).

Each round you must choose one of them clicking the corresponding button.

These are some examples of what you could earn in a round.

Example 1: Imagine you choose GREEN, 3 of your neighbors choose GREEN and 1 chooses BROWN. In that round you will earn $3 \times 7 + 1 \times 0 = 21$ ECUs.

Example 2: In another round you choose BROWN, 2 of your neighbors choose GREEN and 2 choose BROWN. In that round you will earn $2 \times 10 + 2 \times 0 = 20$ ECUs.

Round iteration.

Remember that each part will consist of an undetermined number of rounds.

Each round you will have up to 20 s to choose a color. After these 20 s, if you didn't choose, the system will choose for you.

Whatever happens it will not affect the behavior of the system in the next rounds: you will be able to make your subsequent choices normally. (Don't worry: 20 s are more than enough to make a choice).

The round will not end until all participants have made their choice.

At the end of each round you will see a screen like this one. Your choice (as given by the color) and your earning in this round. Also your [NUMBER_OF_NEIGHBORS] neighbors' choices (represented by their colors) and their respective earnings in that round.

Your neighbors' earnings are given with respect to your number of neighbors. For example, you have 5 neighbors and one is Ferdinand (fictitious name). Ferdinand in turn has two neighbors: one is you and the other a stranger. If Ferdinand has won 10 ECUs in the last round, the gain of Ferdinand that you are shown is: $(10 \text{ ECUs} / 2 \text{ neighbors of Ferdinand}) \times 5 \text{ neighbors of you} = 25 \text{ ECUs}$.

Note that what each of your neighbors has won depends on what you have chosen and also on what the neighbors of your neighbors have chosen.

Immediately after finishing a round there will be a new one, and then another one, and so on until you see a screen warning you about the end of that part of the experiment.

Note that the prompt placeholder [NUMBER_OF_NEIGHBORS] is replaced by the number of neighbors.

Backstory variants

The code supports three backstory type values:

- **STUDENT**: adds “You are a high school student.”
- **GENDER**: adds a “Gender: man/woman.” line (based on vertex property `is_man` from the data of [2]).
- **STUDENT_AND_GENDER**: combines the above.

Round-to-round description

For rounds $r \geq 1$, the agent receives a per-neighbor summary. The description contains:

- The round index.
- Each neighbor’s previous choice and the neighbor’s *normalized* payoff.
- The focal node’s previous choice and payoff.
- The instruction: “Based on this information, make your choice for this round.”

S1.5 Parsing and error handling

- The script expects the agent response to be valid JSON. Because LLMs occasionally output non-strict JSON, the code contains helpers to sanitize and robustly parse responses:
 - *Normalize json string(s)*: trims text outside the outermost `{...}`, removes control characters, attempts to fix common issues (missing braces, extra whitespace between braces, etc.), and finally ensures the string looks like a JSON object.
 - *Convert string to json*: wraps `json` loads and, when necessary, raises a descriptive error on failure.
 - *String to choice output (json string)*: uses the normalizer, parses the JSON, and returns a Python dict with keys `choice` and `reasoning`.

Note that this is not the standard implementation. However, we implemented it this way to avoid minor issues common to less sophisticated LLMs.

- If parsing or the Crew kickoff call fails, the code retries up to `max_tries` times (default `max_tries = 10`). On repeated failure, the implementation falls back to:
 - a default response `{"choice": "GREEN", "reasoning": "Error: ... defaulting to GREEN."}`; or

S1.6 Simulation: main loop and execution logic

The simulation proceeds for `round_num = 0, ..., max_rounds-1`. Each round consists of the following steps:

1. Description preparation.

- For round 1, agents receive a full explanation of the payoff matrix, examples, and the experimental structure.
- For subsequent rounds ($r > 1$), agents receive:
 - each neighbor’s previous choice,
 - each neighbor’s normalized payoff,
 - their own previous choice and payoff.

2. Task construction.

A `Task` object is created for each non-random agent. The task description is injected into the prompt template through the `{input}` placeholder, ensuring that the round-specific information is embedded into a fixed JSON-format instruction.

3. Decision generation.

For each node i :

- If the node is random, a uniform random choice between GREEN and BROWN is generated.

- Otherwise, the CrewAI pipeline is executed:

```
response = Crew(...).kickoff()
parsed = string_to_choice_output(response.raw)
```

4. Retry mechanism and robustness.

LLM calls may fail for two reasons:

- (a) **Server or connection failures** (e.g., temporary unavailability of the Ollama endpoint).
- (b) **Formatting errors** (the LLM does not produce valid JSON).

To ensure robustness, each agent call is retried up to `max_tries = 10` times.

This retry mechanism is important because:

- Server-side failures occasionally occur in long simulations.
- We empirically observed that when formatting errors occur, they are significantly more frequent in the *first round*. In later rounds, once the model has produced a correctly formatted JSON response, formatting mistakes become much less frequent.

If all retries fail, the system assigns a safe default response:

```
{"choice": "GREEN", "reasoning": "Error... defaulting to GREEN."}
```

This ensures that the simulation continues without interruption.

5. **Payoff computation.** After all nodes have chosen payoffs are computed using the pairwise payoff matrix.

High-level structured pseudocode

```
initialize graph(s) g
assign exactly floor(rho * N) random agents
create LLM agents for non-random nodes

for round_num in 0 .. max_rounds-1:
    prepare round-specific description

    for each node i:
        if random:
            result = random_choice()
        else:
            retry up to 10 times:
                call LLM
                attempt JSON parsing
            if all retries fail:
                result = default GREEN

    log result

compute payoffs
store payoffs in the graph
update cooperation statistics
end
```

S2 Quantitative analysis

S2.1 Definitions

- Sources of data:
 - Empirical: E .

- Simulations: S (LLM players and LLM + random players).
- Notation:
 - $\mathcal{D} \in \{E, S\}$: dataset kind
 - c : Experimental condition (student, gender, and student + gender).
 - g : Network structure.
 - N : Number of network nodes.
 - r : Index for number of trials or replications in simulation.
 - i : Agent index.
 - t : Time step.
 - $a_i^{\mathcal{D}}(t)$: action $\{C, D\}$ at time t for dataset $\mathcal{D}$, where C and D are cooperate and defect, respectively.

Let’s define the indicator variable:

$$y_i^{\mathcal{D}}(t) = \mathbf{1} [a_i^{\mathcal{D}}(t) = C] \in \{0, 1\}, \quad (1)$$

thus, $y = 1$ means cooperation.

S2.2 Macro analysis

Cooperation rate (macro): For each condition c , topology g , and trial r , compute:

$$X_{c,g,r}^{\mathcal{D}}(t) = \frac{1}{N} \sum_{i=1}^N y_i^{\mathcal{D}}(t), \quad (2)$$

which is the fraction of cooperators at time t .

Cooperation propensity per agent (micro):

$$P_i^{\mathcal{D}} = \frac{1}{T} \sum_{t=1}^T y_i^{\mathcal{D}}(t), \quad (3)$$

i.e., how often an agent i cooperates over the full temporal horizon T .

S2.2.1 Time evolution analysis

Here, we test whether the model can reproduce the evolution of cooperation over time. To accomplish this, we compare simulated and empirical series. First, we compute the trajectory errors to determine whether they are equal at the same iteration. Then, we examine the temporal dependence structure of the empirical data using an autoregressive model.

S2.2.2 Comparison of trajectory errors

Before using an AR model to examine the temporal dynamics, we assess how closely the simulated series replicates the empirical series in terms of point-by-point accuracy. Two common metrics are used:

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{T} \sum_{t=1}^T (X_{c,g,r}^E(t) - X_{c,g,r}^S(t))^2}, \quad (4)$$

which penalizes larger deviations more heavily.

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{T} \sum_{t=1}^T |X_{c,g,r}^E(t) - X_{c,g,r}^S(t)|, \quad (5)$$

which measures the average absolute deviation and is less sensitive to outliers.

While RMSE and MAE provide a measure of trajectory similarity between the two series, they do not capture the underlying temporal dependence structure. Therefore, we complement this analysis with AR model estimation to assess the persistence properties of the series.

S2.2.3 Correlation analysis

In addition to trajectory errors, we also compute the Pearson correlation coefficient between the empirical and simulated time series. While RMSE and MAE measure the magnitude of deviations at each time step, they do not capture whether the simulated series follows the same temporal pattern as the empirical data. Pearson correlation quantifies the degree of linear association between the two series, indicating whether increases and decreases in cooperation occur synchronously over time. We calculate Pearson both for the raw correlation series (r) and for the first differences of the series (r^Δ) to capture correlations in the changes between consecutive time steps. For each correlation, we report the corresponding p -value to evaluate the statistical significance of the observed association, providing a measure of confidence that the observed correlation is not due to random chance.

S2.2.4 Autoregressive Comparison Between Empirical and Simulated Series

Before estimating the autoregressive model, we verify whether the empirical and simulated time series are stationary. Stationarity ensures that the mean, variance, and autocovariance structure of the series do not change over time, which is a fundamental assumption for AR model estimation. To do this, we apply the Augmented Dickey-Fuller (ADF) test to both series [1]. In our analysis, we estimate the model using the first-differenced series, $\Delta X_{c,g,r}^{\mathcal{D}}(t)$, rather than the level series $X_{c,g,r}^{\mathcal{D}}(t)$. The differenced specification captures changes in cooperation over time and satisfies the stationarity requirement, whereas the level series exhibits non-stationary behavior according to the ADF test.

To assess whether the simulated time series reproduces the temporal dependence structure of the empirical data, we estimate an autoregressive model of order one (AR(1)) for the differenced series [3]. For a series $\Delta X_{c,g,r}^{\mathcal{D}}(t)$, the AR(1) model is given by

$$\Delta X_{c,g,r}^{\mathcal{D}}(t) = \kappa + \phi \Delta X_{c,g,r}^{\mathcal{D}}(t-1) + \varepsilon(t), \quad (6)$$

where κ is a constant, ϕ measures persistence, and $\varepsilon(t)$ is a white noise error term. We estimate the model separately for the empirical differenced series $\Delta X_{c,g,r}^E(t)$ and the simulated series $\Delta X_{c,g,r}^S(t)$. The key parameter of interest is the persistence coefficient ϕ . To evaluate whether the simulation reproduces the dynamic structure, we use the following analysis metrics:

- **Difference in persistence coefficients:** To assess whether the simulated series reproduces the temporal dependence structure of the empirical data, we compare the estimated AR(1) persistence parameters through the absolute difference

$$\text{Diff}_\phi = |\phi_S - \phi_E|. \quad (7)$$

The coefficient ϕ measures the strength and direction of temporal dependence. If $|\phi_S - \phi_E|$ is small, the simulation reproduces the degree of persistence (or anti-persistence) observed in the empirical data. Large differences indicate that the simulated dynamics adjust either too quickly or too slowly relative to the empirical process, even if the overall trajectories appear similar.

- **Implied mean increment:** For a stationary AR(1) process in first differences ($|\phi_{\mathcal{D}}| < 1$), the unconditional mean of the differenced process is

$$\mu_{\mathcal{D}} = \frac{\kappa_{\mathcal{D}}}{1 - \phi_{\mathcal{D}}}. \quad (8)$$

This quantity represents the average change around which the series increments fluctuate over time. Comparing μ_E and μ_S allows us to verify whether simulated and empirical processes exhibit similar average dynamics. Substantial differences indicate systematic biases in the evolution of cooperation, even if persistence properties are similar.

S2.2.5 Null Model

Since the magnitude of the proposed metrics cannot be directly interpreted, we use a null model to evaluate their relevance. The null model generates a synthetic time series of the same length as the empirical and simulated series. Observations are independently drawn from the empirical marginal distribution. This procedure preserves the unconditional distribution of cooperation levels while eliminating any temporal dependence structure.

Let M^{obs} denote the observed value of a metric. To construct a reference distribution under the null hypothesis of no temporal structure, we generate B independent null realizations and compute the corresponding metric values $\{M^{(1)}, \dots, M^{(B)}\}$. Next, the statistical significance is assessed by comparing M^{obs} to the empirical distribution of $\{M^{(b)}\}_{b=1}^B$. Here, we use $B = 10,000$. The null hypothesis is rejected when M^{obs} lies in the tails of the null distribution.

The Monte Carlo two-tailed p -value is defined as

$$p = \frac{1}{B} \sum_{b=1}^B \mathbf{1}(|M^{(b)} - \mu_{\text{null}}| \geq |M^{obs} - \mu_{\text{null}}|), \quad (9)$$

where $\mu_{\text{null}} = \frac{1}{B} \sum_{b=1}^B M^{(b)}$ is the mean of the null distribution. Here, we adopt this approach because a small $p_{\text{two-tailed}}$ indicates that the observed metric is unlikely under the null hypothesis, given deviations in either direction. Note that for Pearson correlation, we adopted the null hypothesis of zero correlation ($H_0 : r = 0$).

We adopt this two-tailed criterion because our goal is to determine whether the observed metric is unusually far from the null expectation, regardless of direction. Under this interpretation, both unusually large and unusually small values of M^{obs} relative to the null distribution are treated as evidence against the null hypothesis. Thus, a small p -value indicates that the observed metric is unlikely under the null model, not only when it exceeds the null expectation, but more generally when it represents an extreme deviation from it.

S2.3 Micro analysis

S2.3.1 Definitions

Switching rate and persistence (micro): Let's define switching rate:

$$S_i^{\mathcal{D}} = \frac{1}{T-1} \sum_{t=2}^T \mathbf{1}[a_i^{\mathcal{D}}(t) \neq a_i^{\mathcal{D}}(t-1)]. \quad (10)$$

Persistence is defined as $\mathcal{P}_i^{\mathcal{D}} = 1 - S_i^{\mathcal{D}}$.

Neighborhood cooperation (for conditional cooperation): For each agent i and for $t \geq 2$, we can calculate the previous round cooperation in the agent's neighborhood $\mathcal{N}$ as

$$m_i^{\mathcal{D}}(t-1) = \frac{1}{|\mathcal{N}_{g_{t-1}}(i)|} \sum_{j \in \mathcal{N}_{g_{t-1}}(i)} y_j^{\mathcal{D}}(t-1), \quad (11)$$

where $\mathcal{N}_{g_{t-1}}(i)$ denotes the set of neighbors of agent i in the interaction graph g_{t-1} . Also for agent i , we can use another variable $z_i^{\mathcal{D}}(t-1) = y_i^{\mathcal{D}}(t-1)$ so that with $z_i^{\mathcal{D}}(t-1)$ and $m_i^{\mathcal{D}}(t-1)$ we can study conditional cooperation.

For the conditional-rule analysis, the p -value is computed using the same Monte Carlo criterion defined in Eq. 9. However, each observation is represented by the triplet $(m_i^{\mathcal{D}}(t-1), z_i^{\mathcal{D}}(t-1), y_i^{\mathcal{D}}(t))$. Under the null hypothesis that $y_i^{\mathcal{D}}(t)$ is independent of the previous-round context, null realizations are generated by randomly permuting the values of $y_i^{\mathcal{D}}(t)$ across observations while keeping $(m_i^{\mathcal{D}}(t-1)$ and $z_i^{\mathcal{D}}(t-1))$ fixed. This preserves the marginal distribution of current actions while removing any association between $y_i^{\mathcal{D}}(t)$ and the predictors $(m_i^{\mathcal{D}}(t-1)$ and $z_i^{\mathcal{D}}(t-1))$.

S2.3.2 Confidence interval

Due to computational constraints, for most tests, only one simulation per condition is performed. In this case, confidence intervals are estimated via bootstrapping across agents rather than simulation runs, capturing uncertainty due to agent heterogeneity.

For the confidence interval (CI), we can use a bootstrap method to set pointwise uncertainty. In practice:

- Sample N agents/players with replacement (i.e., a player can appear multiple times).
- Recompute $\theta^{\mathcal{D}}(t)$ using the sampled agents.

- Repeat B times to obtain bootstrap replicates $\left\{\theta_{c,S}^{\mathcal{D},(b)}(t)\right\}_{b=1}^B$.
- Define $CI_{95\%}(t) = [Q_{0.025}, Q_{0.975}]$.

To quantitatively measure the difference between these trajectories, we use RMSE and MAE to measure the distance between the trajectories $\bar{\theta}_{c,S}^E$ and $\bar{\theta}_{c,S}^S$. In addition, we calculate the Pearson correlation coefficient for both for the raw correlation series (r) and for the first differences of the series (r^Δ) between the trajectories $\bar{\theta}_{c,S}^{E,(b)}$ and $\bar{\theta}_{c,S}^{S,(b)}$ to capture correlations in the changes between consecutive time steps.

For cases where we run the dynamics R times, we use the Cluster bootstrap to account for observations being grouped by simulation run. Specifically, for each bootstrap replicate, we first resample runs with replacement, then resample agents within each selected run. We then recompute the statistic of interest at each timestep. Repeating this many times yields a bootstrap distribution from which we take the percentiles to have the 95% confidence interval. This procedure preserves both within-run dependence and between-run variability, providing more realistic uncertainty than pooling all agents across runs as independent observations.

S2.3.3 Heterogeneity across individuals

To evaluate whether simulations reproduce the diversity of behavioral patterns observed in empirical data, we compare the distributions of individual cooperation propensities across agents. Let

$$\{P_i^E\}_{i=1}^N \quad \text{and} \quad \{P_i^S\}_{i=1}^N$$

denote the cooperation propensities computed for empirical and simulated datasets, respectively. These quantities capture the frequency with which each agent cooperates over the full temporal horizon and therefore encode behavioral heterogeneity at the individual level.

We quantify the discrepancy between both distributions using the first-order Wasserstein distance. First, we sort both samples in non-decreasing order:

$$P_{(1)}^{\mathcal{D}} \leq P_{(2)}^{\mathcal{D}} \leq \dots \leq P_{(N)}^{\mathcal{D}}, \quad \mathcal{D} \in \{E, S\}.$$

The empirical Wasserstein distance of order one between the two samples is then defined as

$$W_1(E, S) = \frac{1}{N} \sum_{k=1}^N \left| P_{(k)}^E - P_{(k)}^S \right|. \quad (12)$$

This metric measures the minimal average amount of probability mass that must be transported to transform the simulated distribution into the empirical one. A small value of $W_1(E, S)$ indicates that the simulation successfully reproduces the level of behavioral heterogeneity observed among real agents, whereas larger values reveal discrepancies such as overly homogeneous or excessively polarized simulated behaviors.

S2.3.4 Conditional cooperation rule

The conditional cooperation rule is defined as the probability that agent i cooperates at time t given its previous action and the cooperation level in its neighborhood:

$$\Pr(y_i^{\mathcal{D}}(t) = 1 \mid m_i^{\mathcal{D}}(t-1), z_i^{\mathcal{D}}(t-1)), \quad (13)$$

recall that

$$z_i^{\mathcal{D}}(t-1) = y_i^{\mathcal{D}}(t-1).$$

Let $\{b_k\}_{k=0}^K$ denote the bin boundaries. Define

$$I_1 = [b_0, b_1], \quad I_k = (b_{k-1}, b_k] \text{ for } k = 2, \dots, K. \quad (14)$$

Then, for each previous action $z \in \{0, 1\}$, define

$$S_{k,z} = \{(i, t) : m_i^{\mathcal{D}}(t-1) \in I_k, z_i^{\mathcal{D}}(t-1) = z\}. \quad (15)$$

The empirical conditional cooperation probability is estimated as

$$\hat{P}_{k,z}^{\mathcal{D}} = \frac{1}{|S_{k,z}|} \sum_{(i,t) \in S_{k,z}} y_i^{\mathcal{D}}(t). \quad (16)$$

Thus, for each bin k and previous action $z \in \{0, 1\}$, we obtain four conditional cooperation probabilities:

$$\hat{P}_{k,0}^E, \hat{P}_{k,1}^E \quad : \text{ for experiments,}$$

$$\hat{P}_{k,0}^S, \hat{P}_{k,1}^S \quad : \text{ for LLMs,}$$

after previous defection ($z = 0$) or cooperation ($z = 1$).

Finally, we can do as before and compare these quantities using RMSE and MAE and Pearson correlation.

S3 Results

S3.1 Visual analysis

First, we show the results for $\rho = 0$.

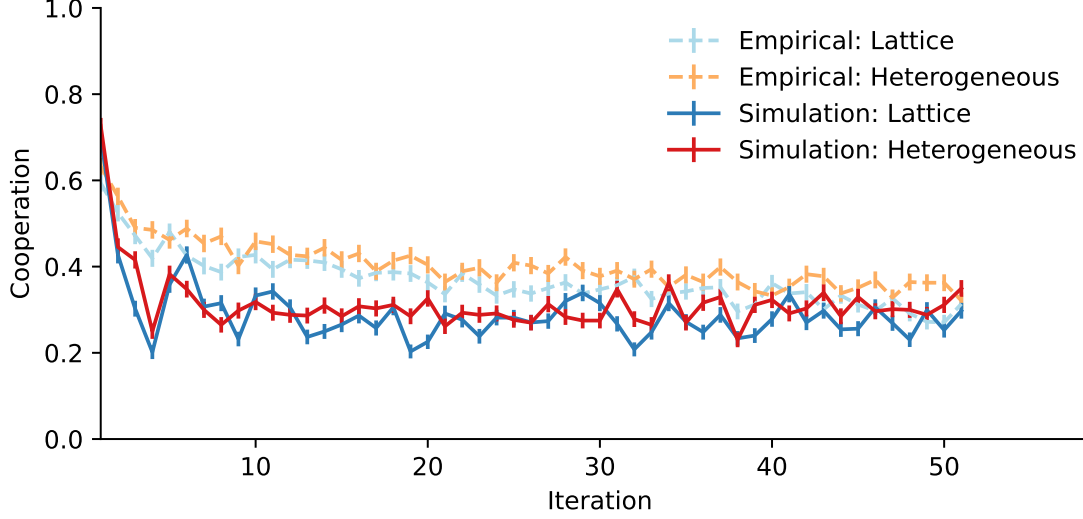


Figure S1: Evolution of cooperation over iterations for the **STUDENT** condition in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4.16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error.

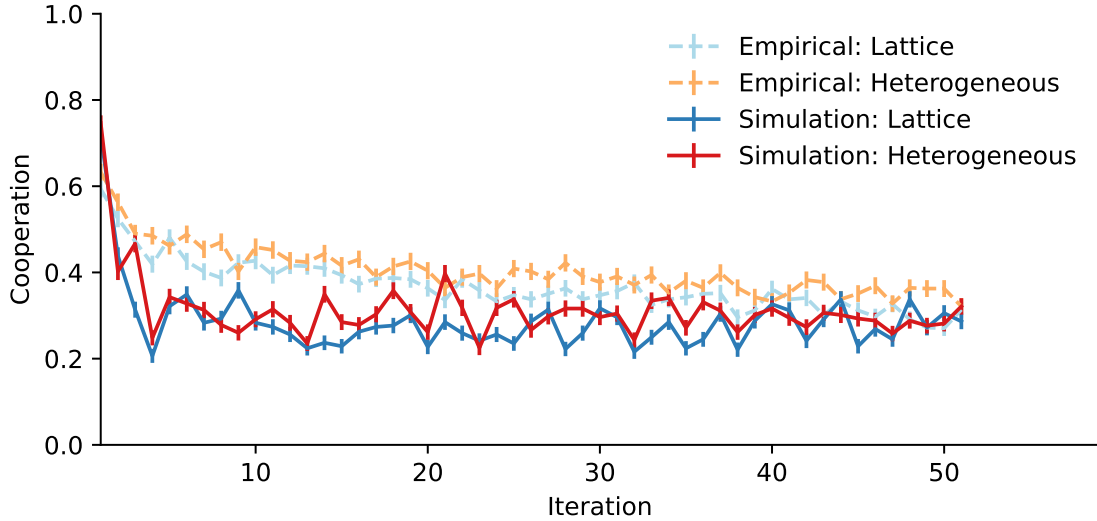


Figure S2: Evolution of cooperation over iterations for the **GENDER** condition in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4.16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error.

Next, we present the results for other values of ρ .

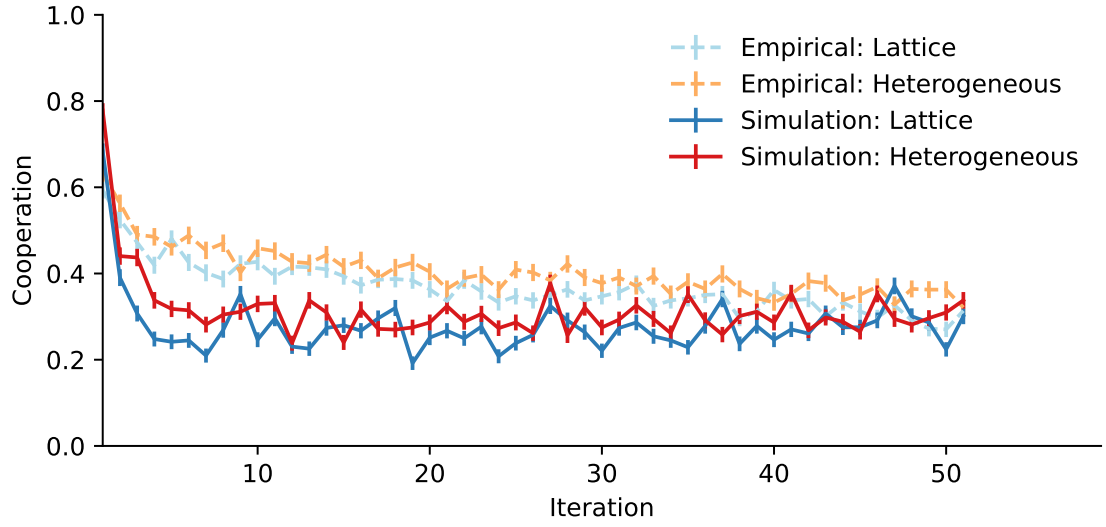


Figure S3: Evolution of cooperation over iterations for the `STUDENT_AND_GENDER` condition in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4_16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error.

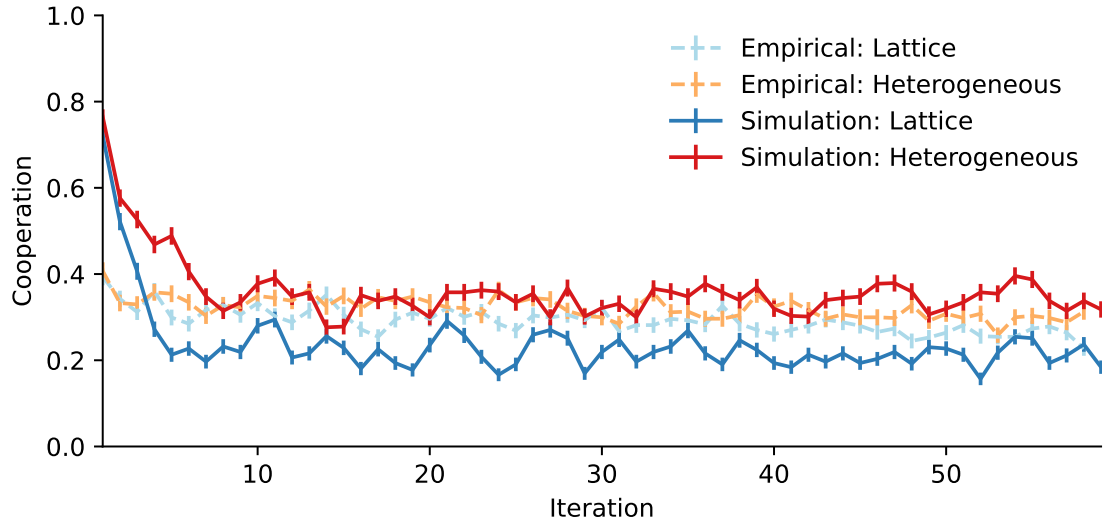


Figure S4: **Control:** Evolution of cooperation over iterations for the `STUDENT` condition in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4_16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error.

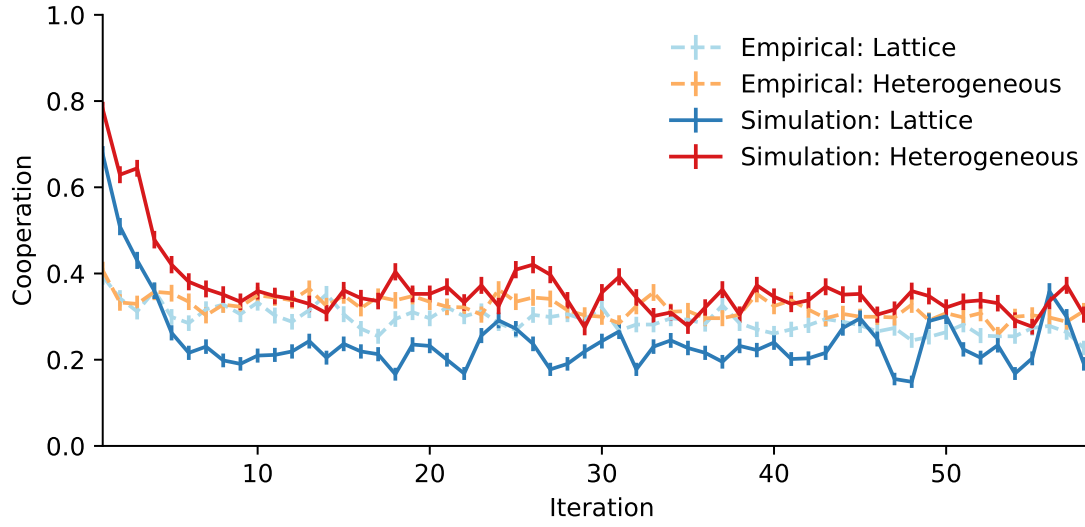


Figure S5: **Control**: Evolution of cooperation over iterations for the **GENDER** condition in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4_16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error.

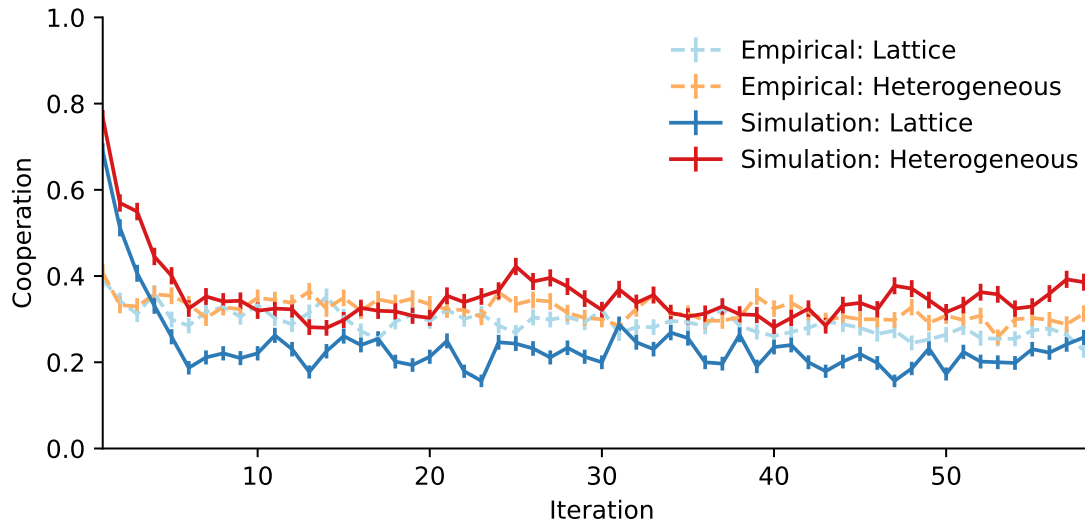


Figure S6: **Control**: Evolution of cooperation over iterations for the **STUDENT_AND_GENDER** condition in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4_16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error.

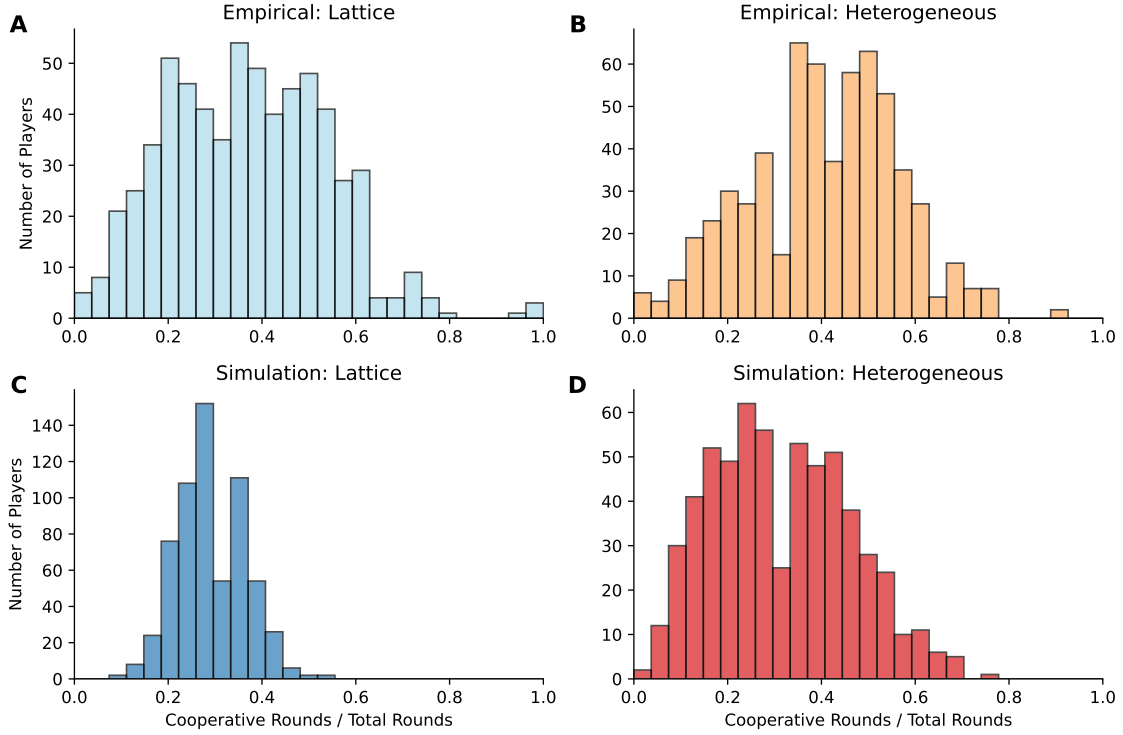


Figure S7: Distribution of per-agent cooperation rates for the **STUDENT** condition setting. Each histogram displays the fraction of cooperative rounds per player. Bins are uniformly spaced in the interval $[0, 1]$. Simulated results are generated using `llama4_16x17b`.

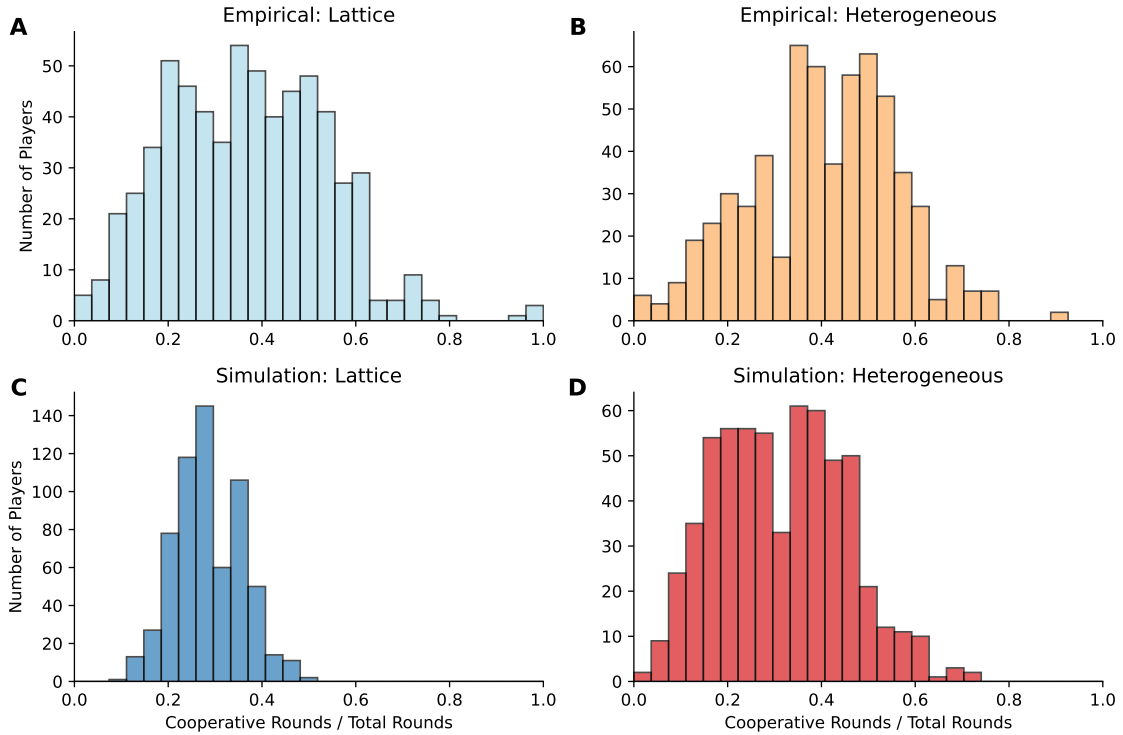


Figure S8: Distribution of per-agent cooperation rates for the **GENDER** condition setting. Each histogram displays the fraction of cooperative rounds per player. Bins are uniformly spaced in the interval $[0, 1]$. Simulated results are generated using `llama4_16x17b`.

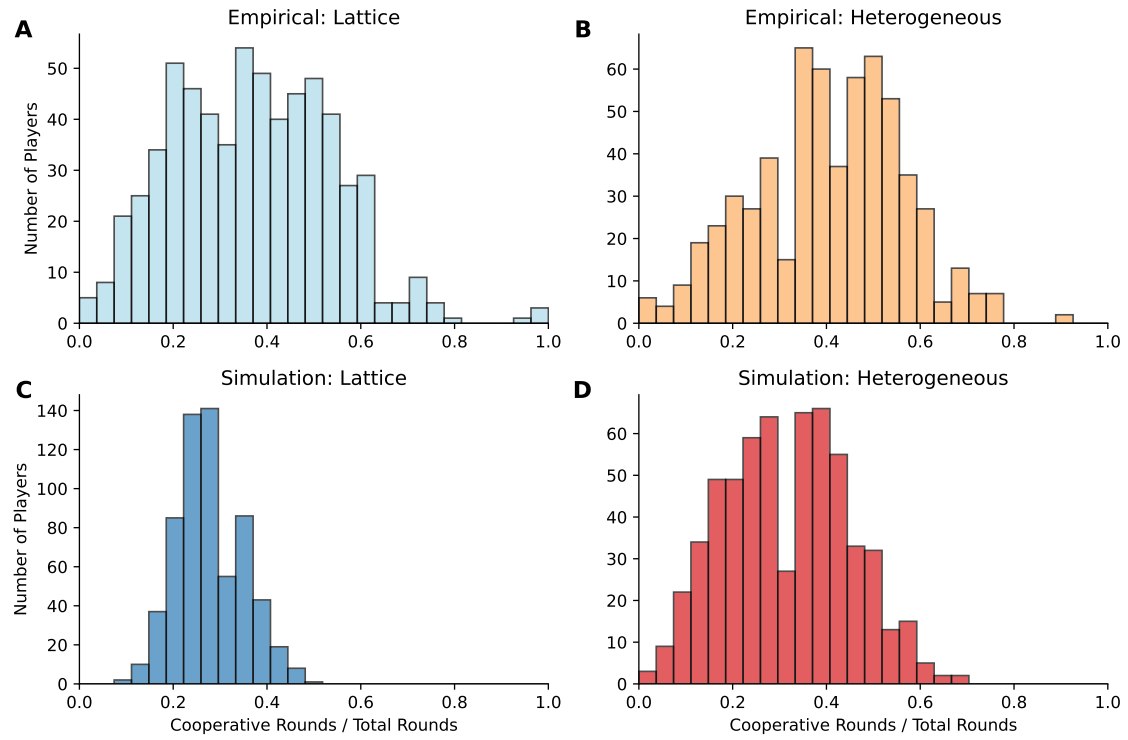


Figure S9: Distribution of per-agent cooperation rates for the **STUDENT_AND_GENDER** condition setting. Each histogram displays the fraction of cooperative rounds per player. Bins are uniformly spaced in the interval $[0, 1]$. Simulated results are generated using **llama4.16x17b**.

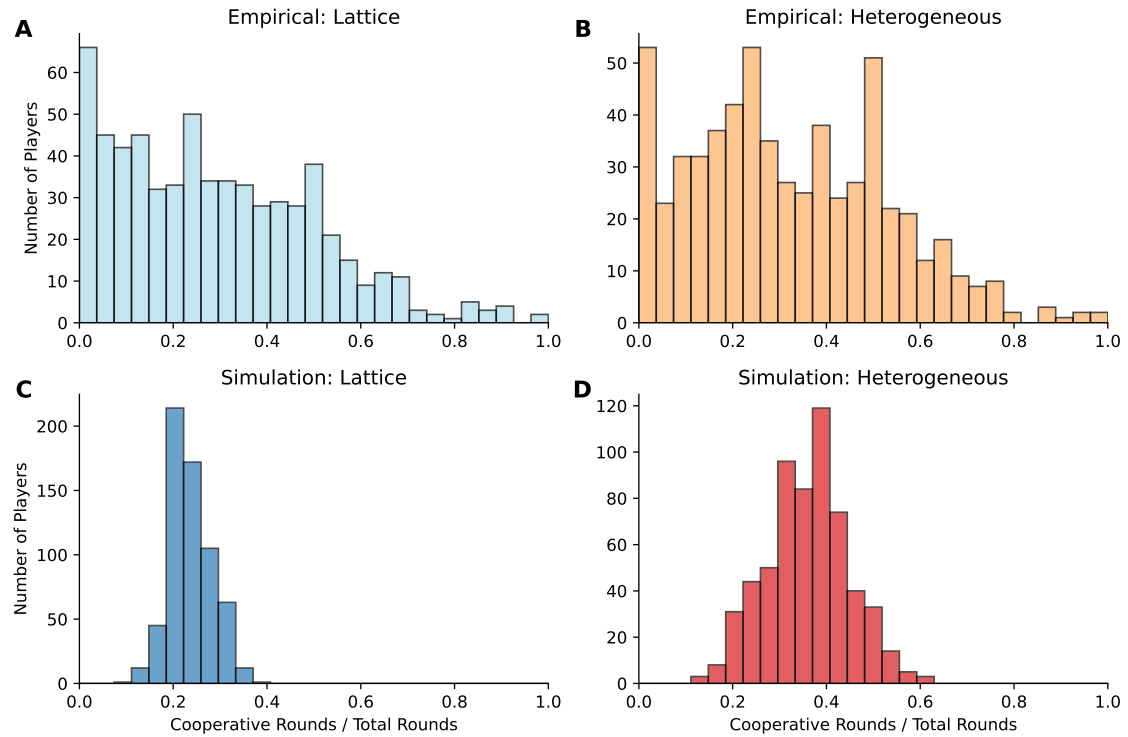


Figure S10: **Control**: Distribution of per-agent cooperation rates for the **STUDENT** condition setting. Each histogram displays the fraction of cooperative rounds per player. Bins are uniformly spaced in the interval $[0, 1]$. Simulated results are generated using **llama4.16x17b**.

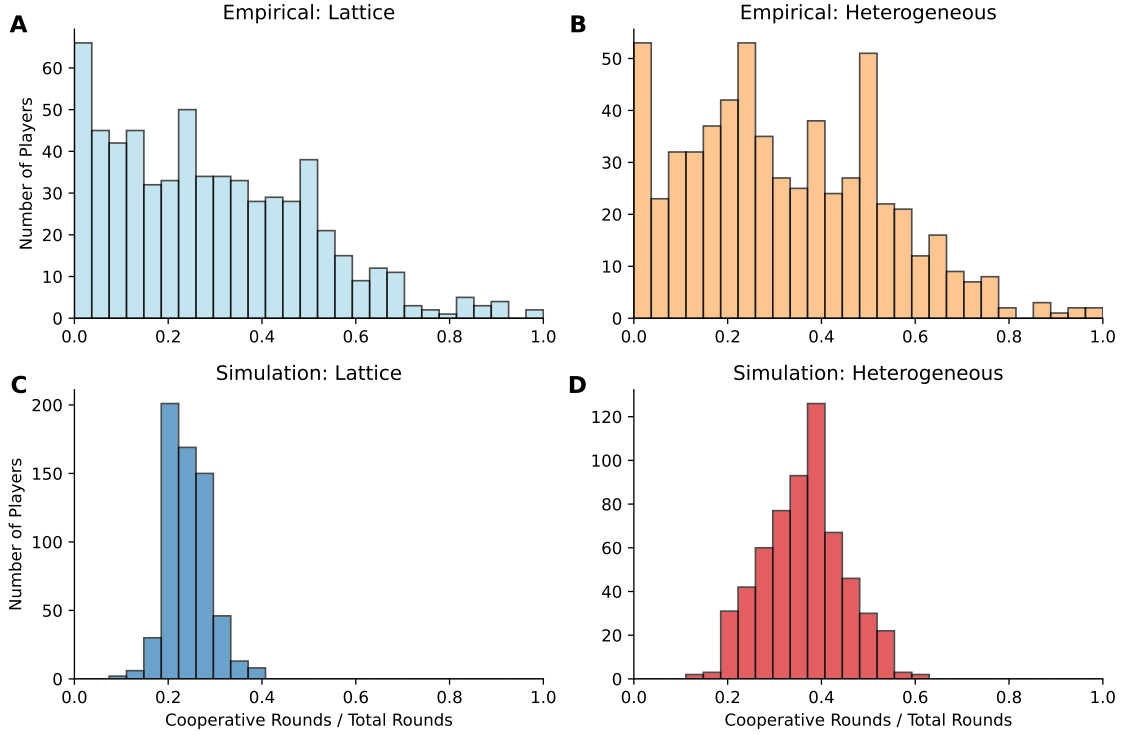


Figure S11: **Control**: Distribution of per-agent cooperation rates for the **GENDER** condition setting. Each histogram displays the fraction of cooperative rounds per player. Bins are uniformly spaced in the interval $[0, 1]$. Simulated results are generated using `llama4.16x17b`.

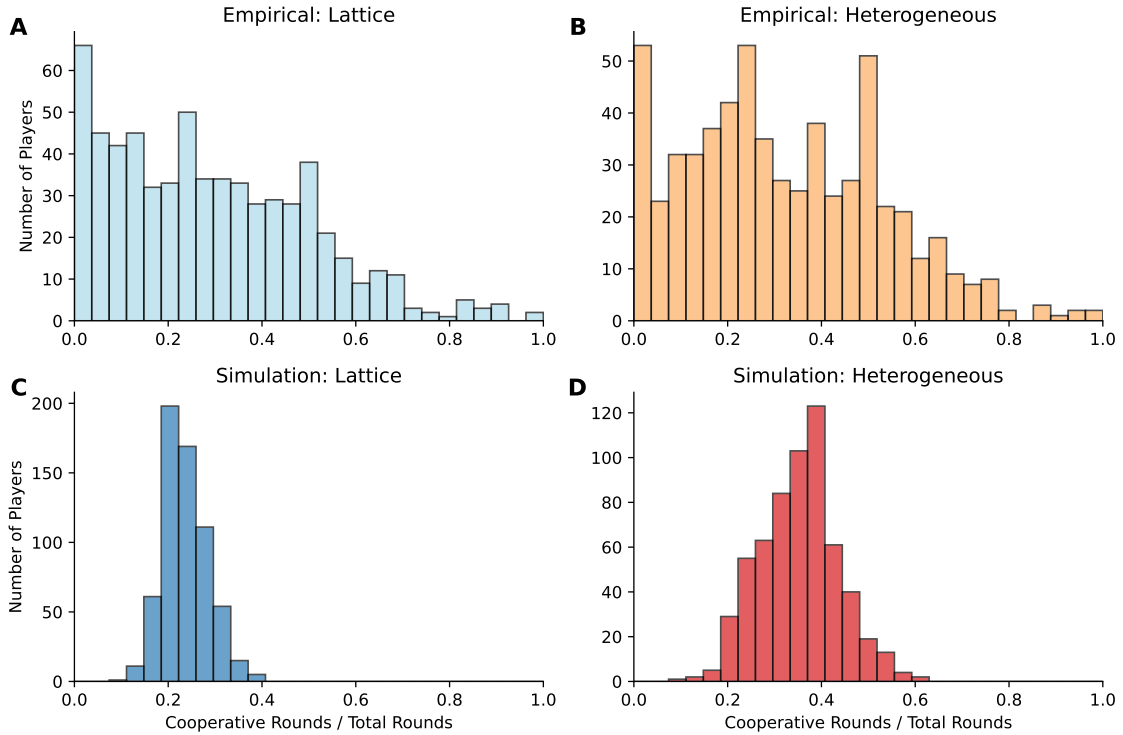


Figure S12: **Control**: Distribution of per-agent cooperation rates for the **STUDENT_AND_GENDER** condition setting. Each histogram displays the fraction of cooperative rounds per player. Bins are uniformly spaced in the interval $[0, 1]$. Simulated results are generated using `llama4.16x17b`.

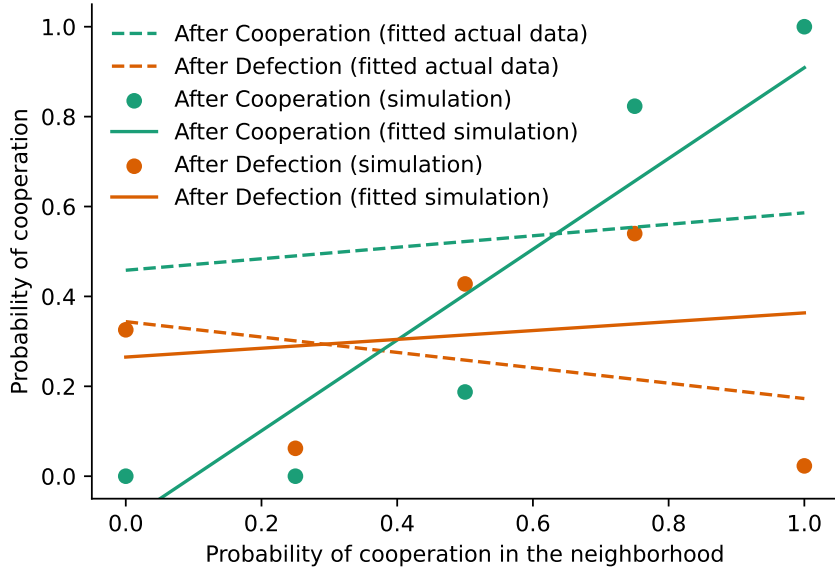


Figure S13: Conditional cooperation as a function of neighborhood cooperation for the `lattice` network under the `STUDENT` condition. The horizontal axis shows the fraction of neighbors who cooperated in the previous round, while the vertical axis shows the probability that the agent cooperates in the current round. Dashed lines denote weighted least-squares fits to the empirical data, where weights equal the number of observations per context. Solid lines denote weighted least-squares fits to the simulation results generated by `llama4.16x17b`.

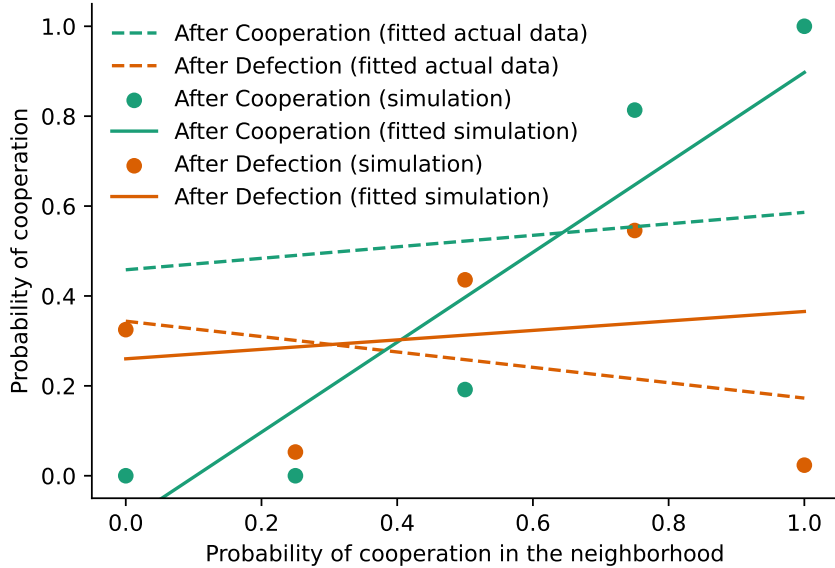


Figure S14: Conditional cooperation as a function of neighborhood cooperation for the `lattice` network under the `GENDER` condition. The horizontal axis shows the fraction of neighbors who cooperated in the previous round, while the vertical axis shows the probability that the agent cooperates in the current round. Dashed lines denote weighted least-squares fits to the empirical data, where weights equal the number of observations per context. Solid lines denote weighted least-squares fits to the simulation results generated by `llama4.16x17b`.

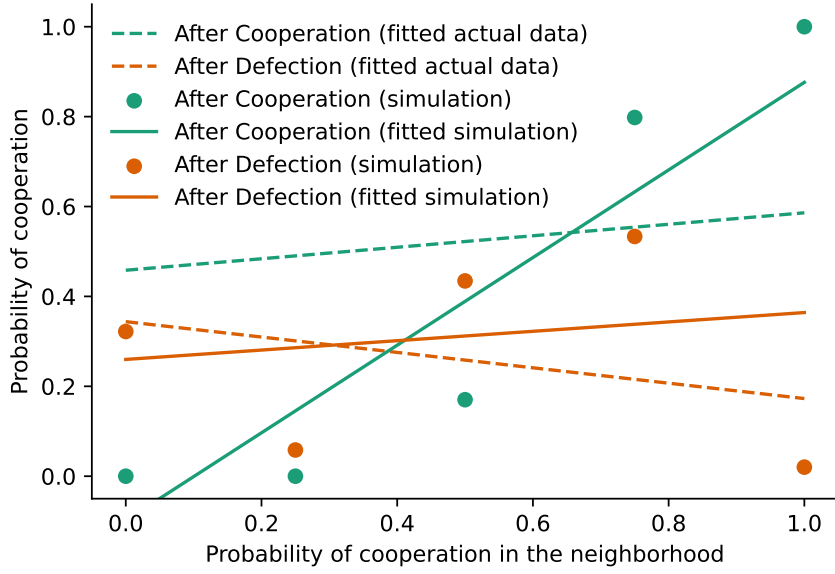


Figure S15: Conditional cooperation as a function of neighborhood cooperation for the `lattice` network under the `STUDENT_AND_GENDER` condition. The horizontal axis shows the fraction of neighbors who cooperated in the previous round, while the vertical axis shows the probability that the agent cooperates in the current round. Dashed lines denote weighted least-squares fits to the empirical data, where weights equal the number of observations per context. Solid lines denote weighted least-squares fits to the simulation results generated by `llama4.16x17b`.

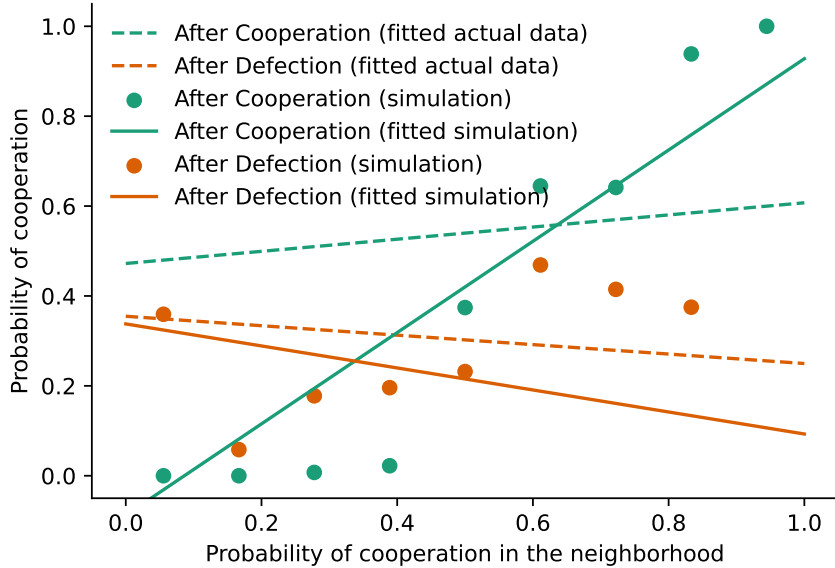


Figure S16: Conditional cooperation as a function of neighborhood cooperation for the `heterogeneous` network under the `STUDENT` condition. The horizontal axis shows the fraction of neighbors who cooperated in the previous round, while the vertical axis shows the probability that the agent cooperates in the current round. Dashed lines denote weighted least-squares fits to the empirical data, where weights equal the number of observations per context. Solid lines denote weighted least-squares fits to the simulation results generated by `llama4.16x17b`.

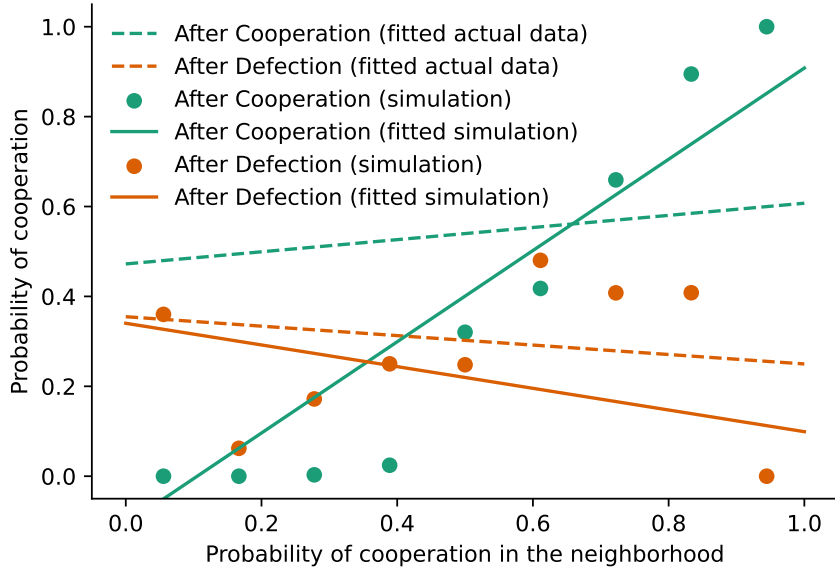


Figure S17: Conditional cooperation as a function of neighborhood cooperation for the **heterogeneous** network under the **GENDER** condition. The horizontal axis shows the fraction of neighbors who cooperated in the previous round, while the vertical axis shows the probability that the agent cooperates in the current round. Dashed lines denote weighted least-squares fits to the empirical data, where weights equal the number of observations per context. Solid lines denote weighted least-squares fits to the simulation results generated by `llama4.16x17b`.

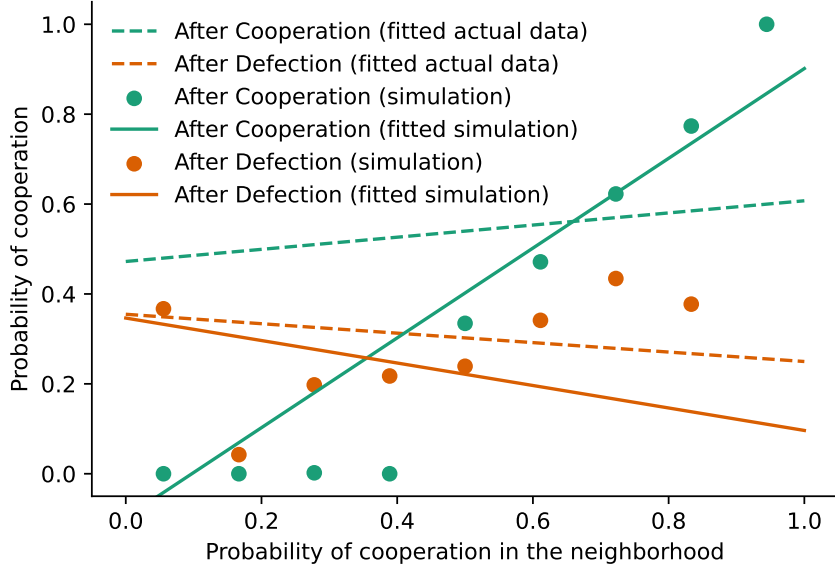


Figure S18: Conditional cooperation as a function of neighborhood cooperation for the **heterogeneous** network under the **STUDENT_AND_GENDER** condition. The horizontal axis shows the fraction of neighbors who cooperated in the previous round, while the vertical axis shows the probability that the agent cooperates in the current round. Dashed lines denote weighted least-squares fits to the empirical data, where weights equal the number of observations per context. Solid lines denote weighted least-squares fits to the simulation results generated by `llama4.16x17b`.

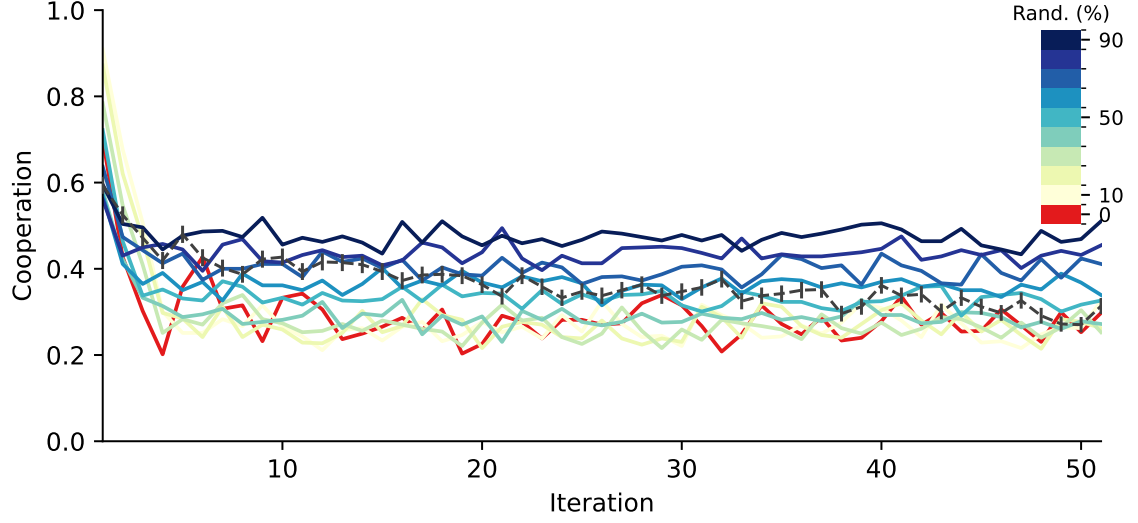


Figure S19: Cooperation over iterations for the `lattice` network, varying ρ , under the `STUDENT` condition. The simulation results were generated by `llama4.16x17b`. The black dashed line shows the empirical data, and the error bars show the standard error.

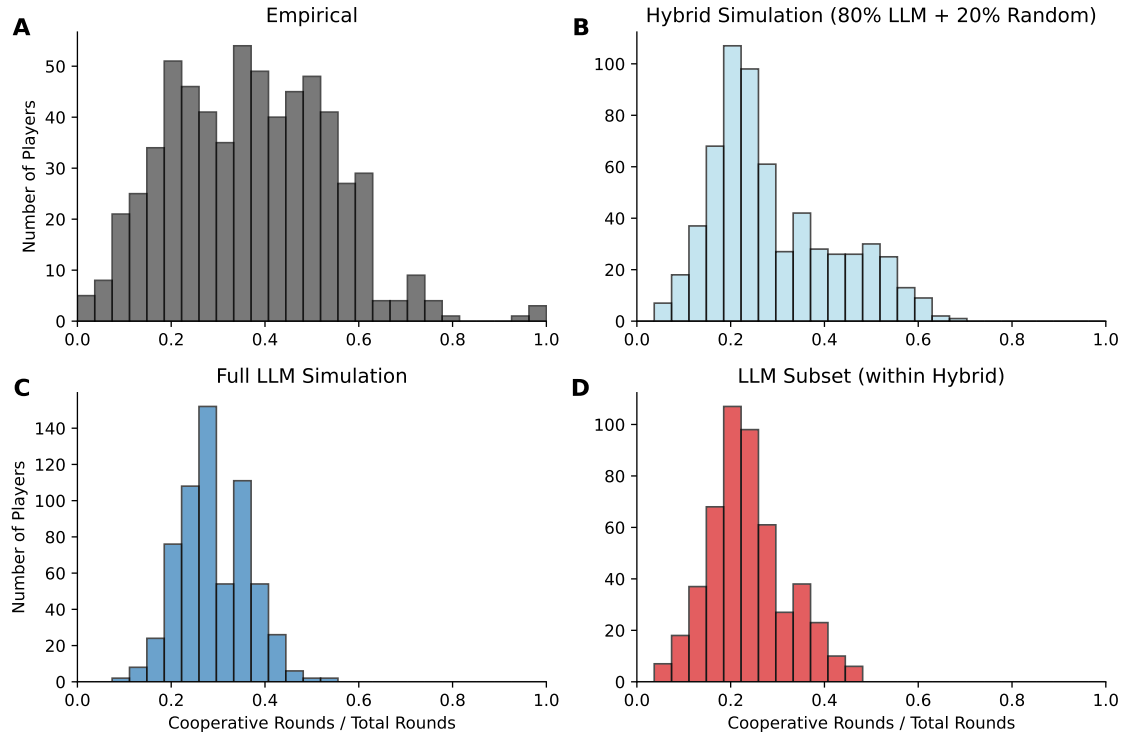


Figure S20: Distribution of per-agent cooperation rates for the `lattice` network with $\rho = 0.2$ (80% LLM + 20% Random), under the `STUDENT` condition. Bins are uniformly spaced in the interval $[0, 1]$. Simulation results were generated using `llama4.16x17b`.

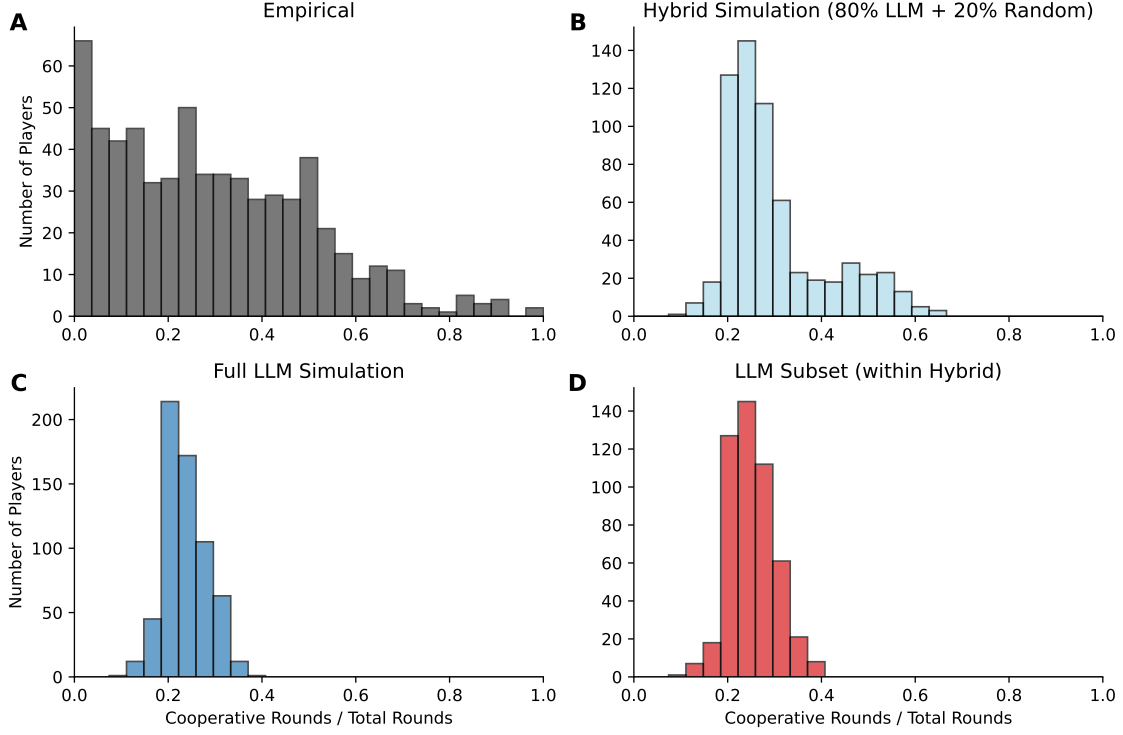


Figure S21: **Control:** Distribution of per-agent cooperation rates for the **lattice control** network with $\rho = 0.2$ (80% LLM + 20% Random), under the **STUDENT** condition. Bins are uniformly spaced in the interval $[0, 1]$. Simulation results were generated using `llama4.16x17b`.

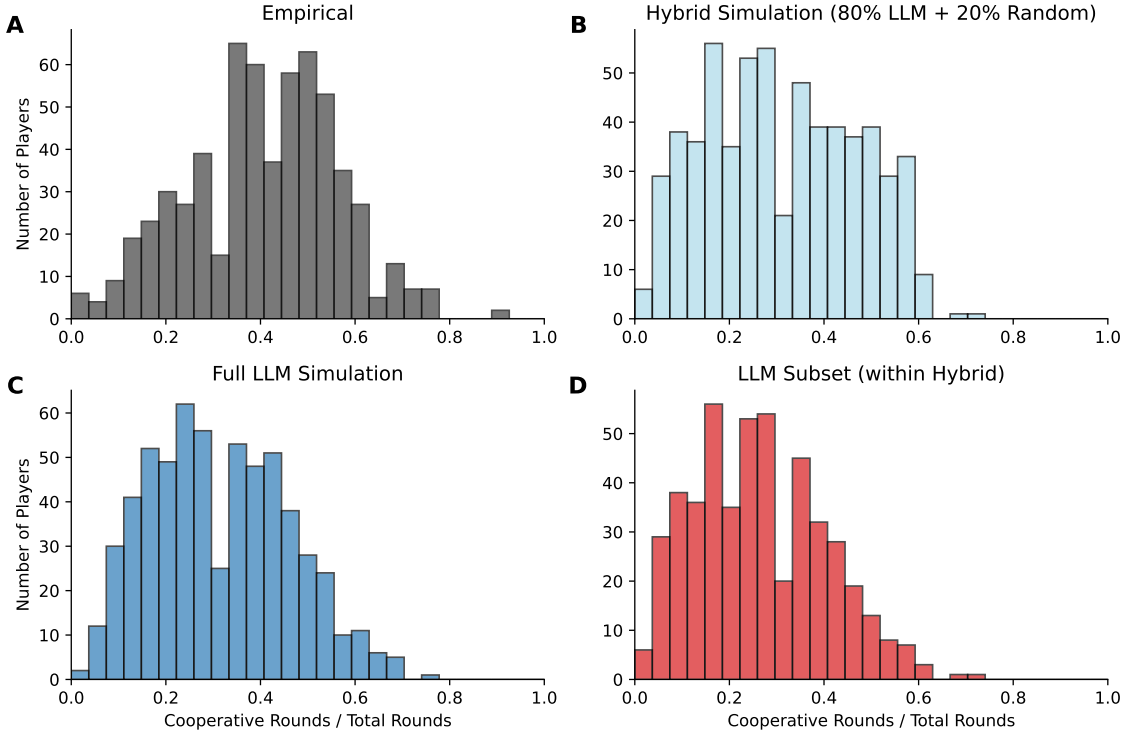


Figure S22: Distribution of per-agent cooperation rates for the **heterogeneous** network with $\rho = 0.2$ (80% LLM + 20% Random), under the **STUDENT** condition. Bins are uniformly spaced in the interval $[0, 1]$. Simulation results were generated using `llama4.16x17b`.

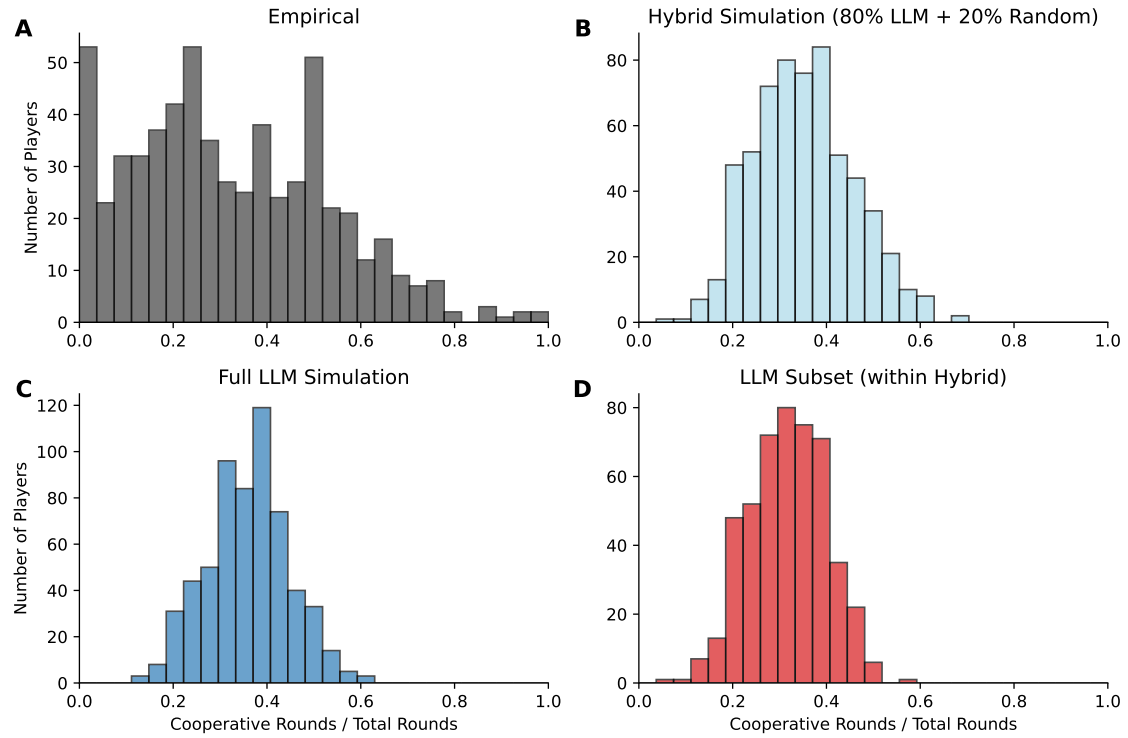


Figure S23: **Control**: Distribution of per-agent cooperation rates for the **heterogeneous control** network with $\rho = 0.2$ (80% LLM + 20% Random), under the **STUDENT** condition. Bins are uniformly spaced in the interval $[0, 1]$. Simulation results were generated using `llama4_16x17b`.

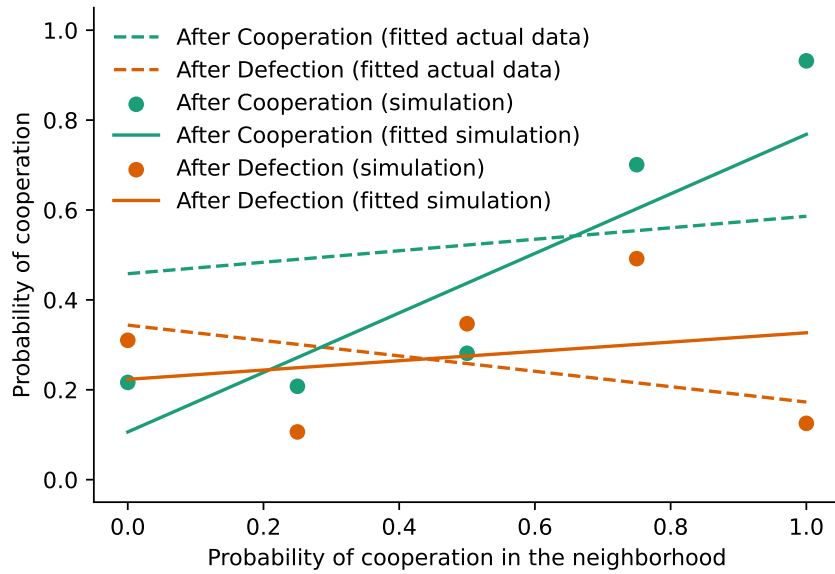


Figure S24: Conditional cooperation as a function of neighborhood cooperation for the **lattice** network under $\rho = 0.2$ (80% LLM + 20% Random). Solid lines show linear fits to the simulated data generated by `llama4_16x17b`, while dashed lines show weighted least-squares fits to the empirical experiment data.

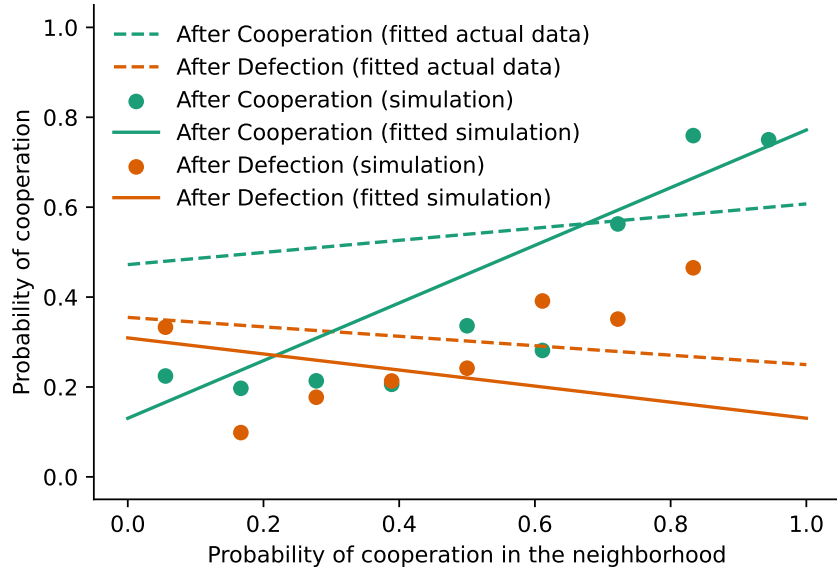


Figure S25: Conditional cooperation as a function of neighborhood cooperation for the **heterogeneous** network under $\rho = 0.2$ (80% LLM + 20% Random). Solid lines show linear fits to the simulated data generated by `llama4_16x17b`, while dashed lines show weighted least-squares fits to the empirical experiment data.

S3.2 Quantitative analysis

S3.2.1 Macro analysis

We begin our analysis by comparing the three **backstory** types, as shown in Figure S26 and Table S1. To reduce the number of simulations, we test two scenarios: (i) Hybrid Simulation (80% LLM + 20% Random) and (ii) Full LLM Simulation.

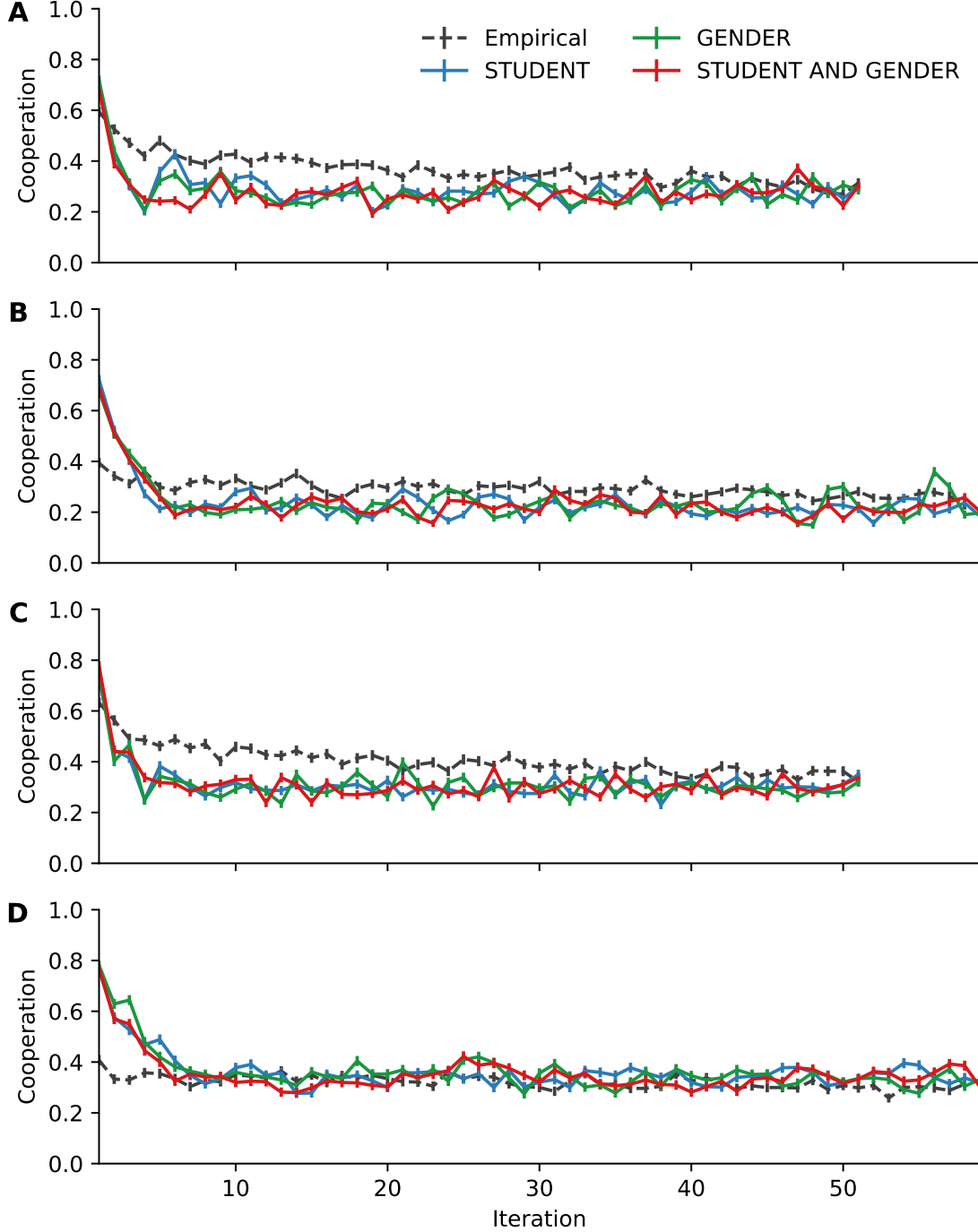


Figure S26: Evolution of cooperation over iterations for the conditions in the setting. Dashed lines show empirical results from the behavioral experiment, while solid lines show simulation results using `llama4_16x17b`. The solid lines represent the mean cooperation rate per round (empirical) or per iteration (simulation), and the error bars represent the standard error. The panels depict the following scenarios: (a) **lattice**, (b) **lattice control**, (c) **heterogeneous**, and (d) **heterogeneous control**.

Experiment	RMSE	MAE	Diff $_{\phi}$	Diff $_{\mu}$	r	r^{Δ}
lattice - GENDER	0.104**	0.090	0.155***	0.001	0.577***	0.367***
lattice - STUDENT	0.098**	0.083**	0.217***	0.002	0.612***	0.276*
lattice - STUDENT AND GENDER	0.110*	0.095	0.113***	0.002	0.495***	0.384***
lattice control - GENDER	0.089***	0.074*	0.383***	0.003***	0.472***	0.151
lattice control - STUDENT	0.089***	0.074*	0.464***	0.002***	0.583***	0.214
lattice control - STUDENT AND GENDER	0.089***	0.075*	0.327***	0.002***	0.506***	0.031
heterogeneous - GENDER	0.110**	0.098	0.017	0.001*	0.606***	0.117
heterogeneous - STUDENT	0.107**	0.096	0.071	0.002	0.622***	0.151
heterogeneous - STUDENT AND GENDER	0.108**	0.097	0.038*	0.001*	0.642***	0.224
heterogeneous control - GENDER	0.087***	0.051*	0.276	0.005*	0.467***	0.002
heterogeneous control - STUDENT	0.079***	0.051	0.359	0.004*	0.440***	0.362***
heterogeneous control - STUDENT AND GENDER	0.078***	0.050	0.420	0.002*	0.367***	0.157

Table S1: Trajectory errors, autoregressive comparison, and correlation across different **backstory types** (for the scenario with no random agents). RMSE and MAE measure point-by-point deviations between empirical and simulated cooperation rates. Diff $_{\phi}$ denotes the absolute difference between the AR(1) persistence coefficients, and Diff $_{\mu}$ denotes the absolute difference between the implied long-run means of the simulated and empirical series. r denotes the Pearson correlation coefficient between empirical and simulated trajectories. For RMSE, MAE, Diff $_{\phi}$, and Diff $_{\mu}$, statistical significance is assessed using the Monte Carlo null model described in the text. For r , significance refers to the null hypothesis of zero correlation ($H_0 : r = 0$). *** denotes $p < 0.01$, ** denotes $p < 0.05$, * denotes $p < 0.1$, and no symbol indicates that the result is not statistically significant.

Since the different **backstory types** do not significantly impact the results, we only consider **STUDENT** for the following results. First, we compare the different executions for both networks. See Figure S27 and Table S2.

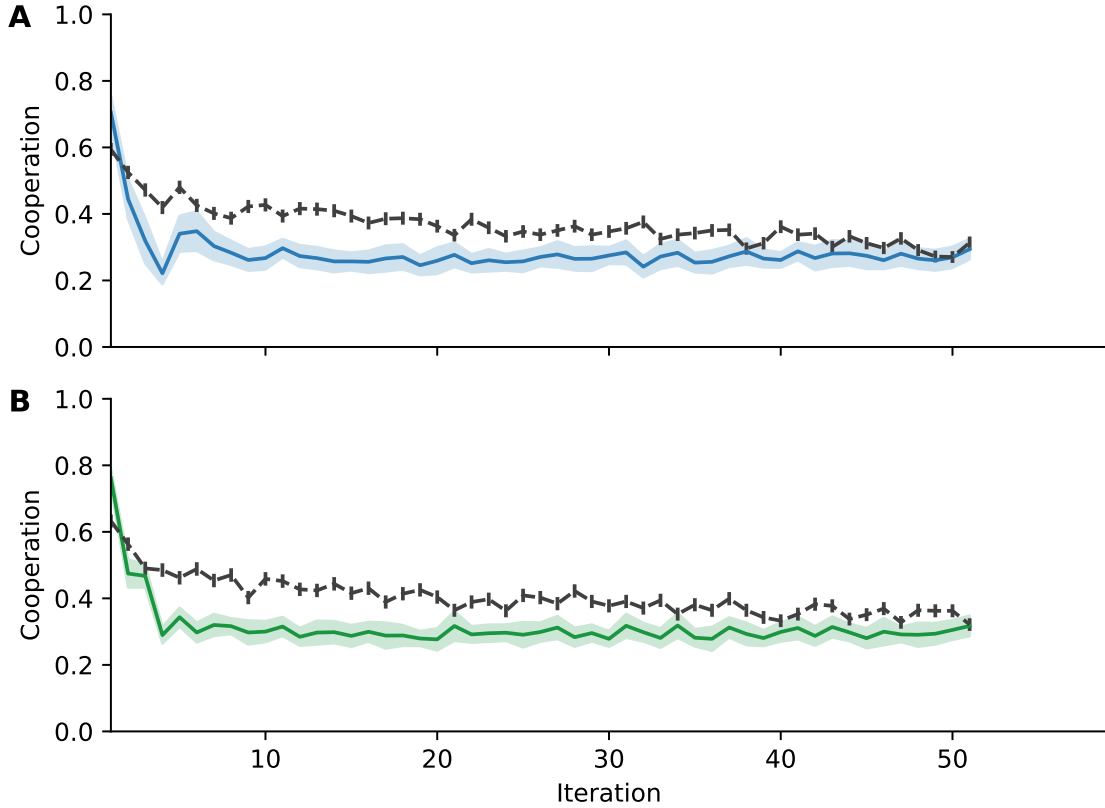


Figure S27: The average evolution of cooperation over $r = 20$ runs for both lattice and heterogeneous networks (panels A and B, respectively). The dashed lines show the results of the behavioral experiment, the error bars represent the standard error, and the solid lines show the simulation results. The shaded regions represent the respective standard deviation.

Network	RMSE	MAE	Diff $_{\phi}$	Diff $_{\mu}$	r	r^{Δ}
Lattice	0.099**	0.088	0.461***	0.004**	0.666***	0.407***
Heterogeneous	0.105**	0.096*	0.190	0.001**	0.708***	0.179

Table S2: Trajectory errors, autoregressive comparison, and correlation across different executions for the scenario with no random agents, on both **lattice** and **heterogeneous** networks. RMSE and MAE measure point-by-point deviations between empirical and average simulated cooperation rates. Diff $_{\phi}$ denotes the absolute difference between the AR(1) persistence coefficients, and Diff $_{\mu}$ denotes the absolute difference between the implied long-run means of the simulated and empirical series. r denotes the Pearson correlation coefficient between empirical and simulated trajectories. For RMSE, MAE, Diff $_{\phi}$, and Diff $_{\mu}$, statistical significance is assessed using the Monte Carlo null model described in the text. For r , significance refers to the null hypothesis of zero correlation ($H_0 : r = 0$). *** denotes $p < 0.01$, ** denotes $p < 0.05$, * denotes $p < 0.1$, and no symbol indicates that the result is not statistically significant.

Table S3 reports trajectory errors, AR(1) differences, and correlation metrics for simulations on **lattice** network, with varying percentages of random agents in the population. Table S4 presents the same metrics across different experimental scenarios (i.e., lattice, heterogeneous, and control setups).

Random (%)	RMSE	MAE	Diff $_{\phi}$	Diff $_{\mu}$	r	r^{Δ}
0	0.098**	0.083**	0.217***	0.002	0.612***	0.276*
10	0.116***	0.099**	0.537***	0.000**	0.665***	0.284**
20	0.109***	0.093**	0.405***	0.001***	0.690***	0.469***
30	0.104***	0.092**	0.443***	0.000***	0.691***	0.384***
40	0.086*	0.074	0.098***	0.001	0.675***	0.282**
50	0.057***	0.044**	0.376***	0.003	0.638***	0.200
60	0.049**	0.037**	0.020***	0.003	0.569***	0.236*
70	0.060***	0.048***	0.058*	0.003	0.645***	0.332**
80	0.091	0.079	0.048*	0.005***	0.304**	-0.097
90	0.121	0.110	0.215*	0.004**	0.457***	0.297**

Table S3: Trajectory errors, autoregressive comparison, and correlation across different percentages of random agents for the same **backstory type** on **lattice** network. RMSE and MAE measure point-by-point deviations between empirical and simulated cooperation rates. Diff $_{\phi}$ denotes the absolute difference between the AR(1) persistence coefficients, and Diff $_{\mu}$ denotes the absolute difference between the implied long-run means of the simulated and empirical series. r denotes the Pearson correlation coefficient between empirical and simulated trajectories. For RMSE, MAE, Diff $_{\phi}$, and Diff $_{\mu}$, statistical significance is assessed using the Monte Carlo null model described in the text. For r , significance refers to the null hypothesis of zero correlation ($H_0 : r = 0$). *** denotes $p < 0.01$, ** denotes $p < 0.05$, * denotes $p < 0.1$, and no symbol indicates that the result is not statistically significant.

Experiment	RMSE	MAE	Diff $_{\phi}$	Diff $_{\mu}$	r	r^{Δ}
lattice (100% LLM + 0% Rand.)	0.098**	0.083**	0.217***	0.002	0.612***	0.276*
lattice (80% LLM + 20% Rand.)	0.109***	0.093**	0.405***	0.001***	0.690***	0.469***
lattice control (100% LLM + 0% Rand.)	0.089***	0.074*	0.464**	0.002***	0.583***	0.214
lattice control (80% LLM + 20% Rand.)	0.087***	0.046**	0.519***	0.005***	0.509***	0.098
heterogeneous (100% LLM + 0% Rand.)	0.107**	0.096	0.071	0.002	0.622***	0.151
heterogeneous (80% LLM + 20% Rand.)	0.109***	0.101**	0.249	0.000***	0.743***	0.343**
heterogeneous control (100% LLM + 0% Rand.)	0.079***	0.051	0.359	0.004*	0.440***	0.362***
heterogeneous control (80% LLM + 20% Rand.)	0.070**	0.046	0.384	0.003*	0.325**	0.135

Table S4: Trajectory errors, autoregressive comparison, and correlation across experimental scenarios. RMSE and MAE measure point-by-point deviations between empirical and simulated cooperation rates. Diff $_{\phi}$ denotes the absolute difference between the AR(1) persistence coefficients, and Diff $_{\mu}$ denotes the absolute difference between the implied long-run means of the simulated and empirical series. r denotes the Pearson correlation coefficient between empirical and simulated trajectories. For RMSE, MAE, Diff $_{\phi}$, and Diff $_{\mu}$, statistical significance is assessed using the Monte Carlo null model described in the text. For r , significance refers to the null hypothesis of zero correlation ($H_0 : r = 0$). *** denotes $p < 0.01$, ** denotes $p < 0.05$, * denotes $p < 0.1$, and no symbol indicates that the result is not statistically significant.

S3.2.2 Micro analysis

First, we show the persistence across different percentages of random agents (in the `lattice` scenario), as shown in Table S5. Next, we present the comparison between the different experiments (see Table S6).

Experiment	$\bar{\mathcal{P}}^{\mathcal{D}}$	95% CI
Empirical	0.640	(0.629, 0.652)
Lattice (100% LLM + 0% Rand.)	0.579	(0.573, 0.586)
Lattice (90% LLM + 10% Rand.)	0.613	(0.605, 0.621)
Lattice (80% LLM + 20% Rand.)	0.633	(0.624, 0.642)
Lattice (70% LLM + 30% Rand.)	0.645	(0.634, 0.656)
Lattice (60% LLM + 40% Rand.)	0.650	(0.637, 0.662)
Lattice (50% LLM + 50% Rand.)	0.610	(0.598, 0.621)
Lattice (40% LLM + 60% Rand.)	0.593	(0.582, 0.605)
Lattice (30% LLM + 70% Rand.)	0.564	(0.554, 0.574)
Lattice (20% LLM + 80% Rand.)	0.540	(0.531, 0.549)
Lattice (10% LLM + 90% Rand.)	0.518	(0.511, 0.525)

Table S5: Mean cooperation persistence and 95% confidence interval for different percentages of random agents. Confidence intervals are estimated via bootstrapping over agents.

Experiment	$\bar{\mathcal{P}}^{\mathcal{D}}$	95% CI
lattice (100% LLM + 0% Rand.)	0.579	(0.573, 0.586)
lattice (80% LLM + 20% Rand.)	0.633	(0.623, 0.642)
lattice - Empirical	0.640	(0.629, 0.651)
lattice control (100% LLM + 0% Rand.)	0.610	(0.604, 0.615)
lattice control (80% LLM + 20% Rand.)	0.589	(0.582, 0.596)
lattice control - Empirical	0.724	(0.711, 0.738)
heterogeneous (100% LLM + 0% Rand.)	0.604	(0.592, 0.617)
heterogeneous (80% LLM + 20% Rand.)	0.629	(0.615, 0.642)
heterogeneous - Empirical	0.620	(0.610, 0.630)
heterogeneous control (100% LLM + 0% Rand.)	0.578	(0.572, 0.584)
heterogeneous control (80% LLM + 20% Rand.)	0.577	(0.570, 0.584)
heterogeneous control - Empirical	0.693	(0.680, 0.707)

Table S6: Mean cooperation persistence and 95% confidence interval. Confidence intervals are estimated via bootstrapping over agents.

Next, we examine conditional cooperation as a function of neighborhood behavior. Figures S28 and S29 show the average probability of cooperation as a function of the fraction of cooperating neighbors in the previous round, separately after a previous defection and after a previous cooperation. Both tested scenarios (`lattice` and `heterogeneous`) comprise 20 dynamics executions.

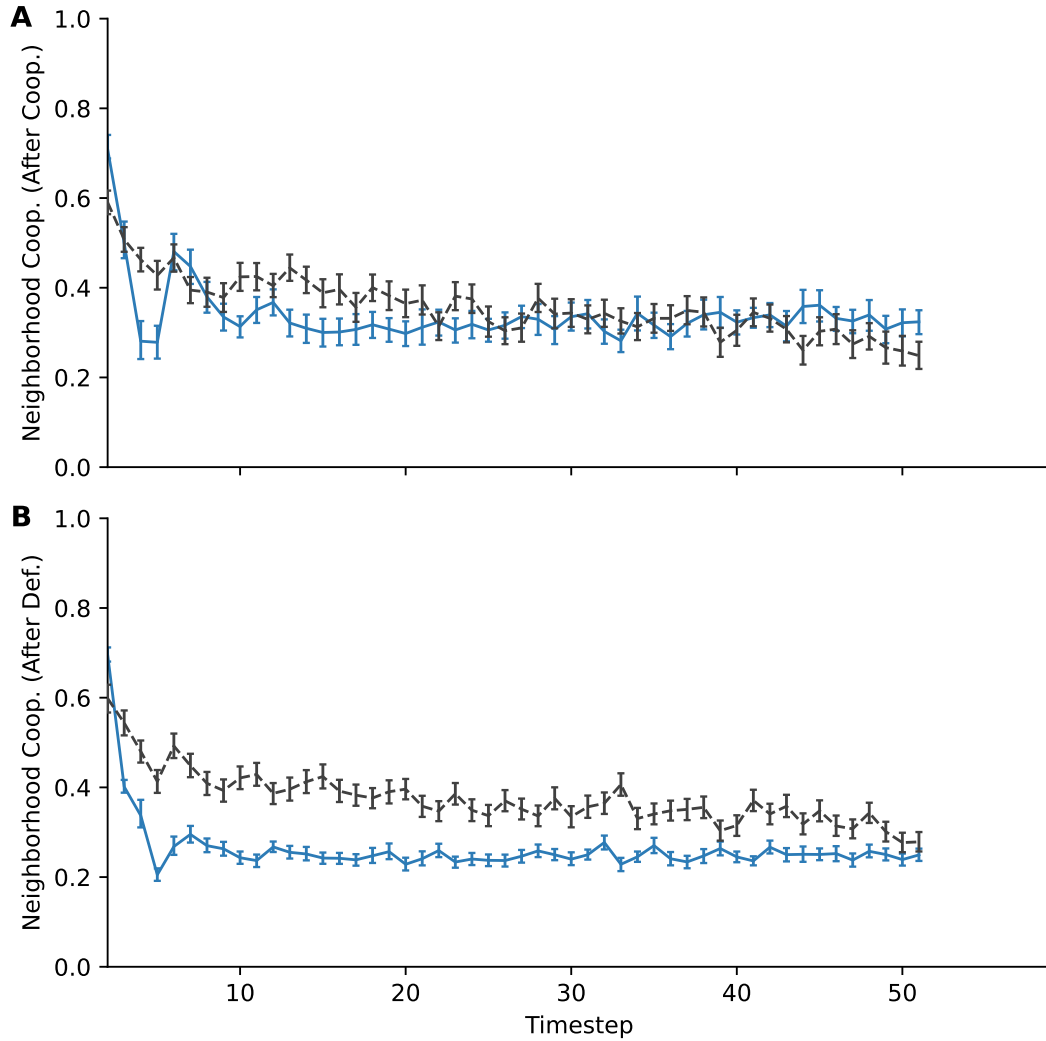


Figure S28: Average fraction of cooperating neighbors over time for the `lattice` network. Results are shown separately for agents that cooperated (panel A) or defected (panel B) in the previous round. The continuous blue line represents the simulation, and the black dashed line represents the empirical data. The simulated curve is the mean across all 20 runs, and the error bars represent 95% CIs estimated by Cluster bootstrap across runs. Empirical patterns (dashed) are shown with 95% CIs estimated by bootstrapping over players.

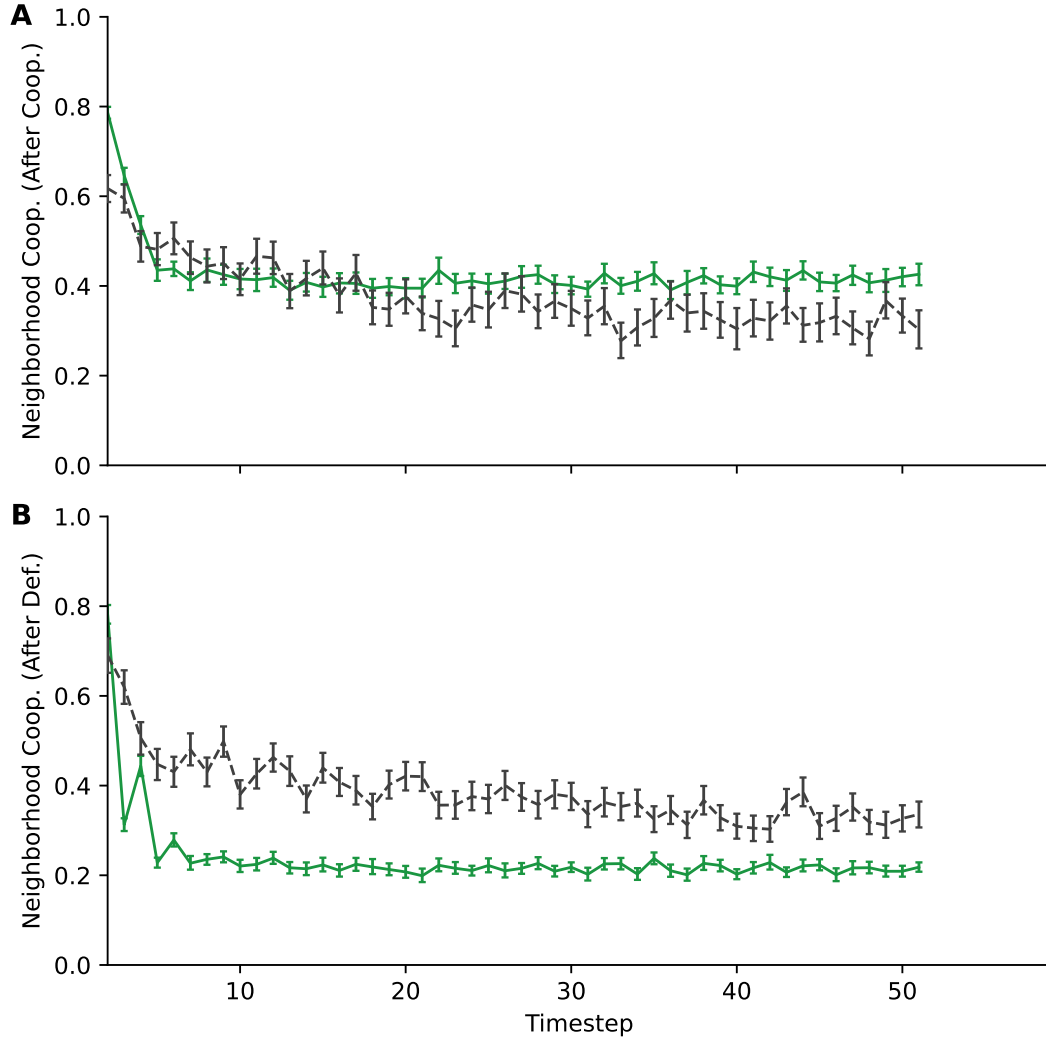


Figure S29: Average fraction of cooperating neighbors over time for the **heterogeneous** network. Results are shown separately for agents that cooperated (panel A) or defected (panel B) in the previous round. The continuous blue line represents the simulation, and the black dashed line represents the empirical data. The simulated curve is the mean across all 20 runs, and the error bars represent 95% CIs estimated by Cluster bootstrap across runs. Empirical patterns (dashed) are shown with 95% CIs estimated by bootstrapping over players.

S3.2.3 Heterogeneity across individuals

To further analyze the role of stochastic behavior in shaping individual-level heterogeneity, we vary the fraction of random agents ρ in the population. Table S7 reports the resulting values of $W_1(E, S)$ across different percentages of random agents. The results show a non-monotonic relationship between ρ and heterogeneity mismatch, with intermediate levels of randomness generally yielding lower distances between empirical and simulated distributions.

Random (%)	$W_1(E, S)$
0	0.101
10	0.089
20	0.081
30	0.084
40	0.073
50	0.037
60	0.039
70	0.051
80	0.080
90	0.119

Table S7: Sensitivity of individual cooperation heterogeneity to the fraction of random agents. Wasserstein distance between empirical and simulated distributions of individual cooperation propensities. Lower values indicate better reproduction of behavioral heterogeneity.

S3.2.4 Conditional cooperation rule

To quantify how well the simulations reproduce the empirical conditional cooperation rule, we compute RMSE and MAE between the empirical and simulated conditional cooperation probabilities. Errors are calculated separately for cases in which the previous action was defection ($z = 0$) and cooperation ($z = 1$), as well as aggregated across both cases, see Table S8.

Random (%)	RMSE _{$z=0$}	MAE _{$z=0$}	RMSE _{$z=1$}	MAE _{$z=1$}
0	0.237**	0.217***	0.397***	0.388*
10	0.208**	0.188***	0.322***	0.310*
20	0.180	0.165***	0.255***	0.246*
30	0.153	0.144***	0.199***	0.192*
40	0.145**	0.136***	0.117***	0.116***
50	0.140***	0.121***	0.096***	0.092***
60	0.137***	0.112***	0.060***	0.055***
70	0.135***	0.101***	0.049***	0.041***
80	0.150***	0.116***	0.042***	0.038***
90	0.181***	0.163***	0.035***	0.027***

Table S8: Deviation between empirical and simulated conditional cooperation rules across different percentages of random agents. Errors are computed separately for cases in which the previous action was defection ($z = 0$) or cooperation ($z = 1$). The columns RMSE_{all} and MAE_{all} summarize the overall deviation across both cases. *** denotes $p < 0.01$, ** denotes $p < 0.05$, * denotes $p < 0.1$, and no symbol indicates that the result is not statistically significant.

References

- [1] David A Dickey and Wayne A Fuller. Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American statistical association*, 74(366a):427–431, 1979.
- [2] Carlos Gracia-Lázaro, Alfredo Ferrer, Gonzalo Ruiz, Alfonso Tarancón, José A Cuesta, Angel Sánchez, and Yamir Moreno. Heterogeneous networks do not promote cooperation when humans play a prisoner’s dilemma. *Proceedings of the National Academy of Sciences*, 109(32):12922–12926, 2012.
- [3] James D. Hamilton. *Time Series Analysis*. Princeton University Press, Princeton, NJ, 1994.